

هوش مصنوعی و کارگاه

دانشکده ریاضی و علوم کامپیوتر دانشگاه صنعتی امیرکبیر



مدرس کلاس: مهدی قطعی

استاد گروه علوم کامپیوتر

www.aut.ac.ir

مدرس کارگاه: بهنام یوسفی مهر

دانشجوی دکتری علوم کامپیوتر

Artificial Intelligence & Workshop

شبکه های عصبی پایه (شبکه های یک و
چندلایه، پرسپترون، شبکه های شعاعی، شبکه
های یادگیری تشدید)

Neural Networks for Machine Learning

Geoffrey Hinton

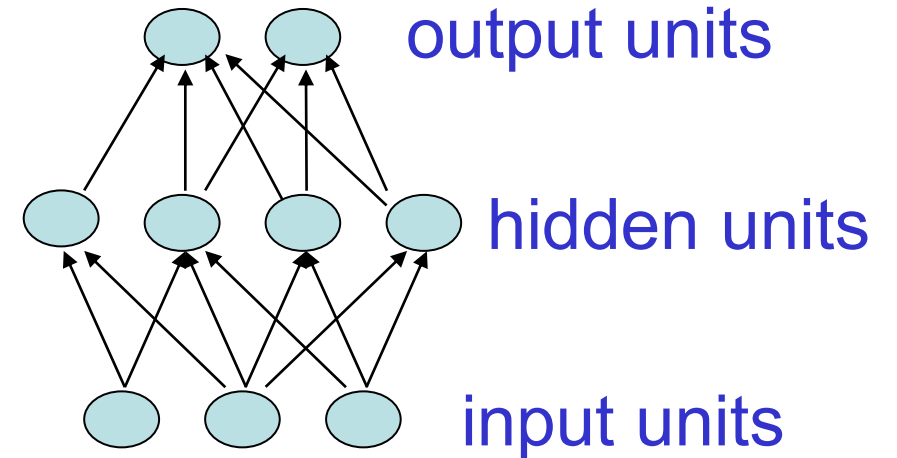
with

Nitish Srivastava

Kevin Swersky

Feed-forward neural networks

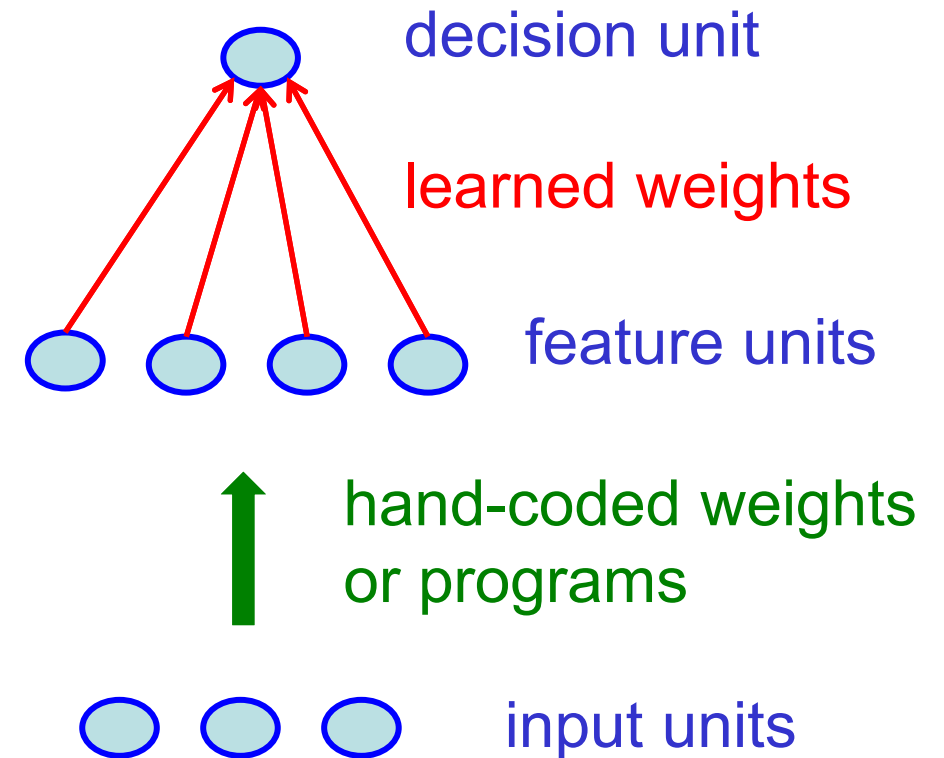
- These are the commonest type of neural network in practical applications.
 - The first layer is the input and the last layer is the output.
 - If there is more than one hidden layer, we call them “deep” neural networks.
- They compute a series of transformations that change the similarities between cases.
 - The activities of the neurons in each layer are a non-linear function of the activities in the layer below.



The standard paradigm for statistical pattern recognition

1. Convert the raw input vector into a vector of feature activations.
Use hand-written programs based on common-sense to define the features.
2. **Learn** how to weight each of the feature activations to get a single scalar quantity.
3. If this quantity is above some threshold, decide that the input vector is a positive example of the target class.

The standard Perceptron architecture



Perceptron

The history of perceptrons

- They were popularised by Frank Rosenblatt in the early 1960's.
 - They appeared to have a very powerful learning algorithm.
 - Lots of grand claims were made for what they could learn to do.
- In 1969, Minsky and Papert published a book called “Perceptrons” that analysed what they could do and showed their limitations.
 - Many people thought these limitations applied to all neural network models.
- The perceptron learning procedure is still widely used today for tasks with enormous feature vectors that contain many millions of features.

A single perceptron

- A **perceptron** is the simplest building block, inspired by a biological neuron.
- **Power:** It is a **binary linear classifier**.
- **What it can do:** It can only solve problems where the data is **linearly separable**. This means you can draw a single straight line (or a hyperplane in higher dimensions) to perfectly separate all the data points of one class from the other.
- **Simple Example: The AND Gate**

x_1	x_2	Output
0	0	0
0	1	0
1	0	0
1	1	1

A single perceptron can learn the correct weights and a bias to create this decision boundary. For example, it could learn:
"Output 1 if $1 \cdot x_1 + 1 \cdot x_2 - 1.5 > 0$,
else output 0

A single perceptron

- A **perceptron** is the simplest building block, inspired by a biological neuron.
- **Power:** It is a **binary linear classifier**.
- **What it can do:** It can only solve problems where the data is **linearly separable**. This means you can draw a single straight line (or a hyperplane in higher dimensions) to perfectly separate all the data points of one class from the other.
- **Limitation:** The famous limitation of a single perceptron is the XOR problem.

x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	0

If you plot these points, you'll find it's **impossible** to draw a single straight line to separate the classes (0s from 1s). The problem is **non-linearly separable**.



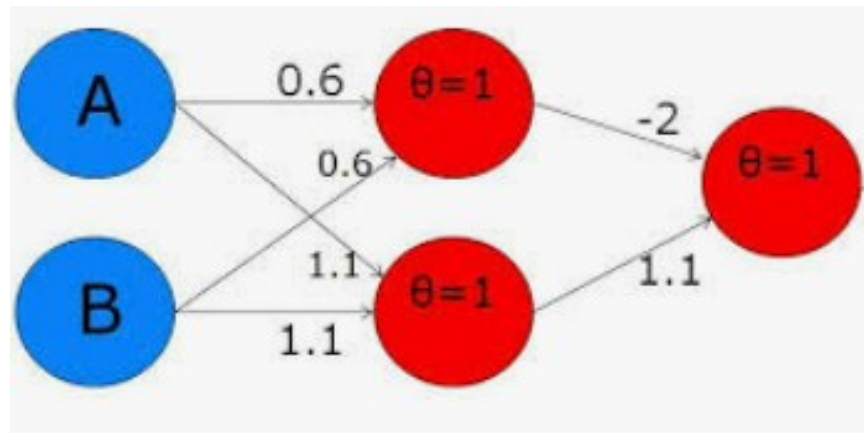
A layer of perceptrons

- **A Layer of Perceptrons (Single Hidden Layer)**
- This is also known as a single-layer neural network (though this can be confusing; it has one *hidden* layer of perceptrons).
- **Power:** It is a **universal function approximator**. This is a monumental leap in power! Given enough perceptrons in the hidden layer, this network can approximate *any* continuous function to an arbitrary level of accuracy, in theory.
- **How it works:** Each perceptron in the hidden layer can learn a different "stripe" or "decision boundary" in the input space. By combining these outputs, the network can create complex, non-linear decision regions.
- **Simple Example: Solving the XOR Gate**
 1. **Perceptron A:** Learns to fire (output 1) if x_1 is 0 and x_2 is 1. It creates one decision boundary.
 2. **Perceptron B:** Learns to fire (output 1) if x_1 is 1 and x_2 is 0. It creates another decision boundary.
 3. **Output Perceptron:** Learns to fire only if *either* A or B is firing. This combines the two linear boundaries to create a non-linear solution.
- The hidden layer transforms the non-linearly separable data in the input space into a new representation that *is* linearly separable in the output layer's space.
- In this transformed space, the output neuron can easily draw a line to separate the classes.

A layer of perceptrons

- **Simple Example: Solving the XOR Gate**

1. **Perceptron A:** Learns to fire (output 1) if x_1 is 0 and x_2 is 1. It creates one decision boundary.
 2. **Perceptron B:** Learns to fire (output 1) if x_1 is 1 and x_2 is 0. It creates another decision boundary.
 3. **Output Perceptron:** Learns to fire only if *either* A or B is firing. This combines the two linear boundaries to create a non-linear solution.
- The hidden layer transforms the non-linearly separable data in the input space into a new representation that *is* linearly separable in the output layer's space.
 - In this transformed space, the output neuron can easily draw a line to separate the classes.

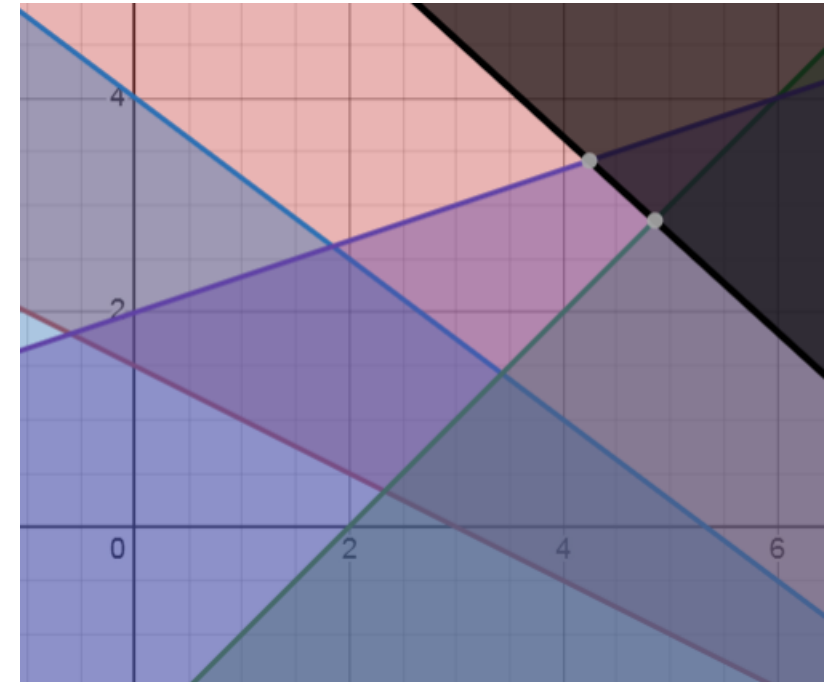
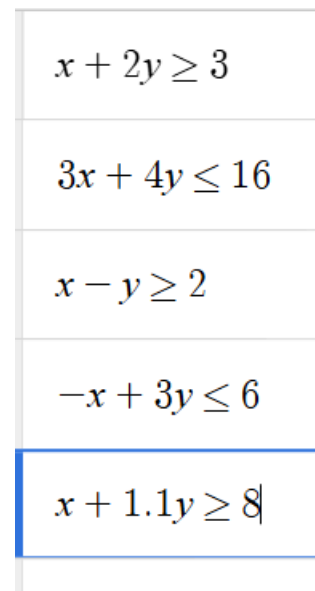
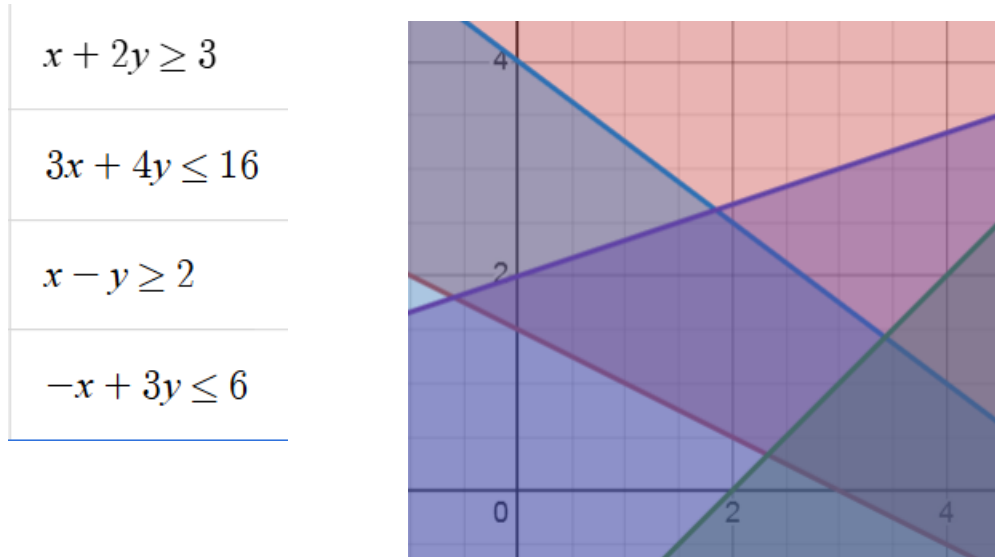


MultiLayer Perceptron (MLP)

- An MLP has one or more hidden layers.
- **Power:** It is also a universal function approximator, but it can often achieve this **more efficiently**. While a single hidden layer is theoretically sufficient, using multiple layers allows the network to learn a **hierarchy of features**, which is much more powerful for complex real-world data.
- **How it works:** Each layer builds on the features learned by the previous one.
 - **Layer 1:** Might learn simple edges and blobs (in image data) or basic patterns (in other data).
 - **Layer 2:** Combines these edges to learn simple shapes like corners and circles.
 - **Layer 3:** Combines these shapes to learn object parts (e.g., an eye, a wheel).
 - **Output Layer:** Combines the object parts to recognize a whole object (e.g., a face, a car).

MultiLayer Perceptron (MLP)

- **How it works:** Each layer builds on the features learned by the previous one.
 - **Layer 1:** Might learn simple edges and blobs (in image data) or basic patterns (in other data).
 - **Layer 2:** Combines these edges to learn simple shapes like corners and circles.
 - **Layer 3:** Combines these shapes to learn object parts (e.g., an eye, a wheel).
 - **Output Layer:** Combines the object parts to recognize a whole object (e.g., a face, a car).



MultiLayer Perceptron (MLP)

- **How it works:** Each layer builds on the features learned by the previous one.
 - **Layer 1:** Might learn simple edges and blobs (in image data) or basic patterns (in other data).
 - **Layer 2:** Combines these edges to learn simple shapes like corners and circles.
 - **Layer 3:** Combines these shapes to learn object parts (e.g., an eye, a wheel).
 - **Output Layer:** Combines the object parts to recognize a whole object (e.g., a face, a car).

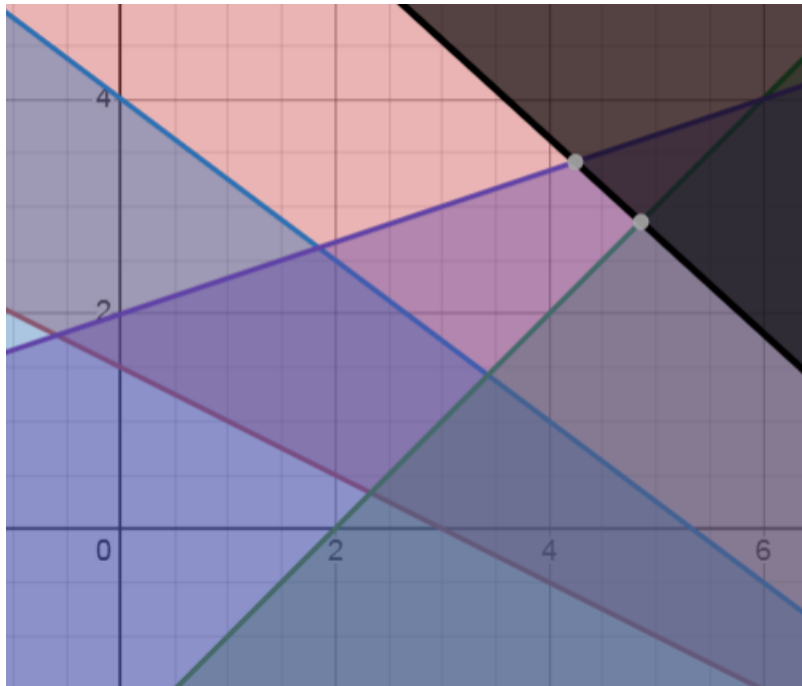
$$x + 2y \geq 3$$

$$3x + 4y \leq 16$$

$$x - y \geq 2$$

$$-x + 3y \leq 6$$

$$x + 1.1y \geq 8$$



$$x + 2y \geq 3$$

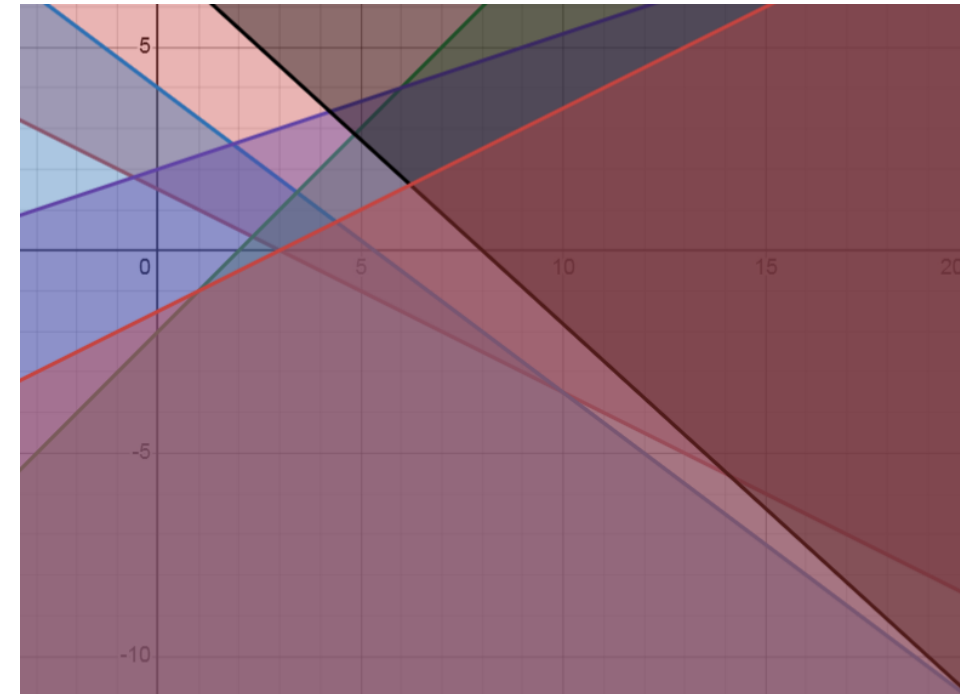
$$3x + 4y \leq 16$$

$$x - y \geq 2$$

$$-x + 3y \leq 6$$

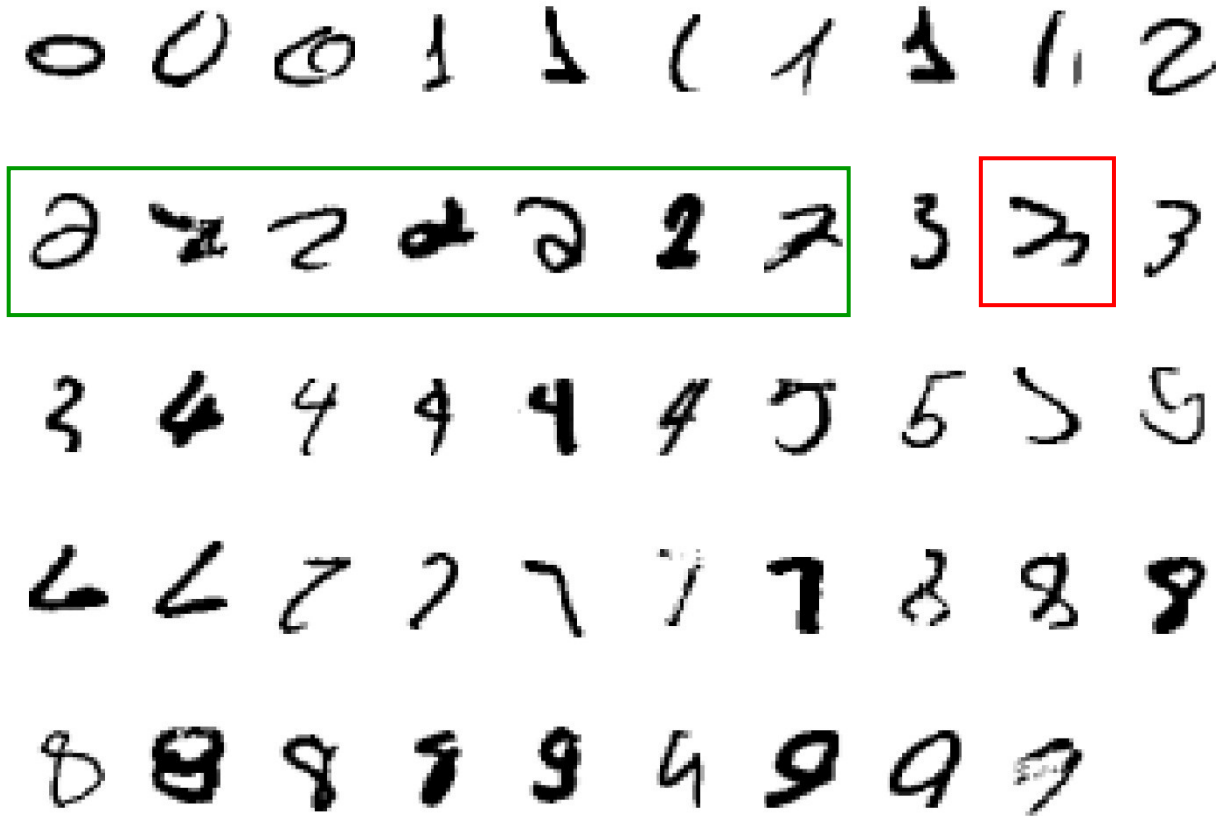
$$x + 1.1y \geq 8$$

$$-x + 2y \leq -3$$



MultiLayer Perceptron (MLP)

- An MLP has one or more hidden layers.
- **Simple Example: Classifying Digits (like MNIST)**
- Let's say we want to classify images of handwritten digits (0-9).
- **Input Layer:** 784 perceptrons (one for each pixel in a 28x28 image).
- **Hidden Layer 1:** Learns to detect low-level features. One neuron might activate for a vertical stroke in the top-left, another for a horizontal stroke in the center, another for a loop.
- **Hidden Layer 2:** Learns to combine these features. One neuron might activate when it sees the features that make up a '9' (a loop at the top and a vertical stroke on the right), another for the features of a '4' (three straight lines meeting at sharp angles).
- **Output Layer (10 neurons):** Each neuron corresponds to a digit (0-9). The neuron that receives the strongest signal from the previous layer's complex feature detectors "wins" and is the network's prediction.
- An MLP can learn this hierarchical feature decomposition, making it vastly more efficient and powerful for such a complex task than a network with just one hidden layer would be.



Examples of handwritten digits that can be recognized
correctly the first time they are seen

0 0 0 1 1 1 1 1 1 2

2 2 2 2 2 2 2 3 3 3

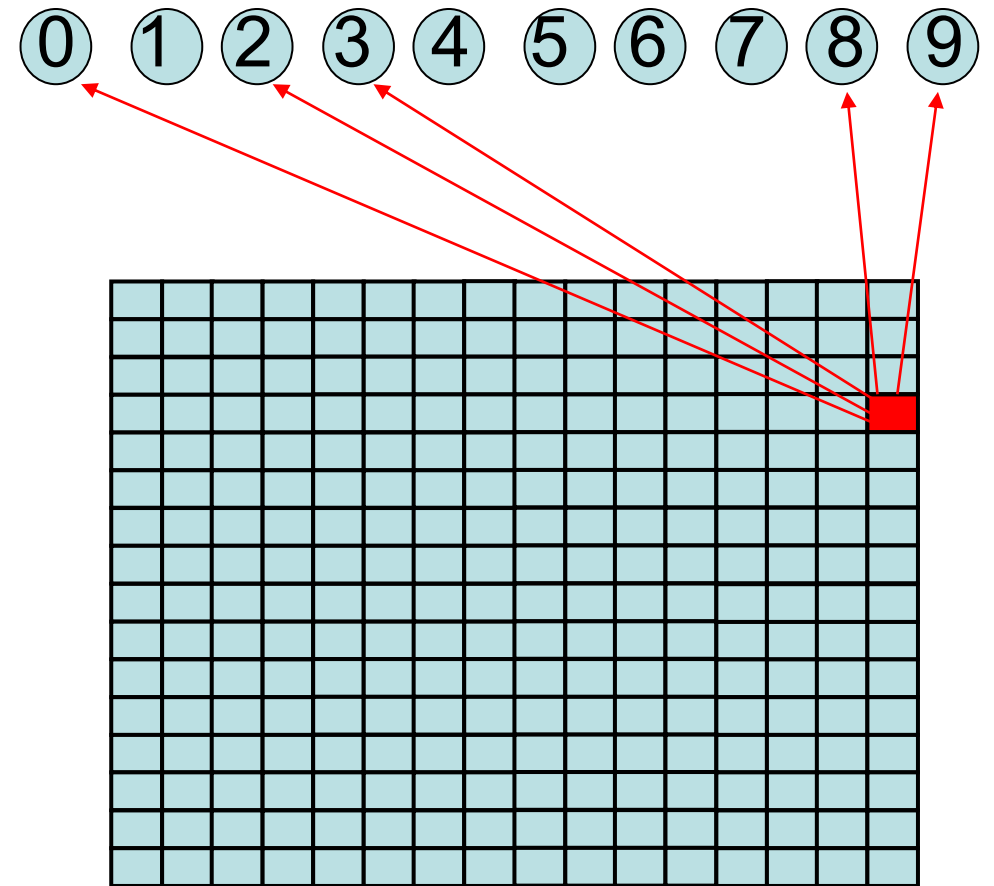
3 4 4 4 4 4 5 5 5 5

6 6 7 7 7 7 8 8 8 8

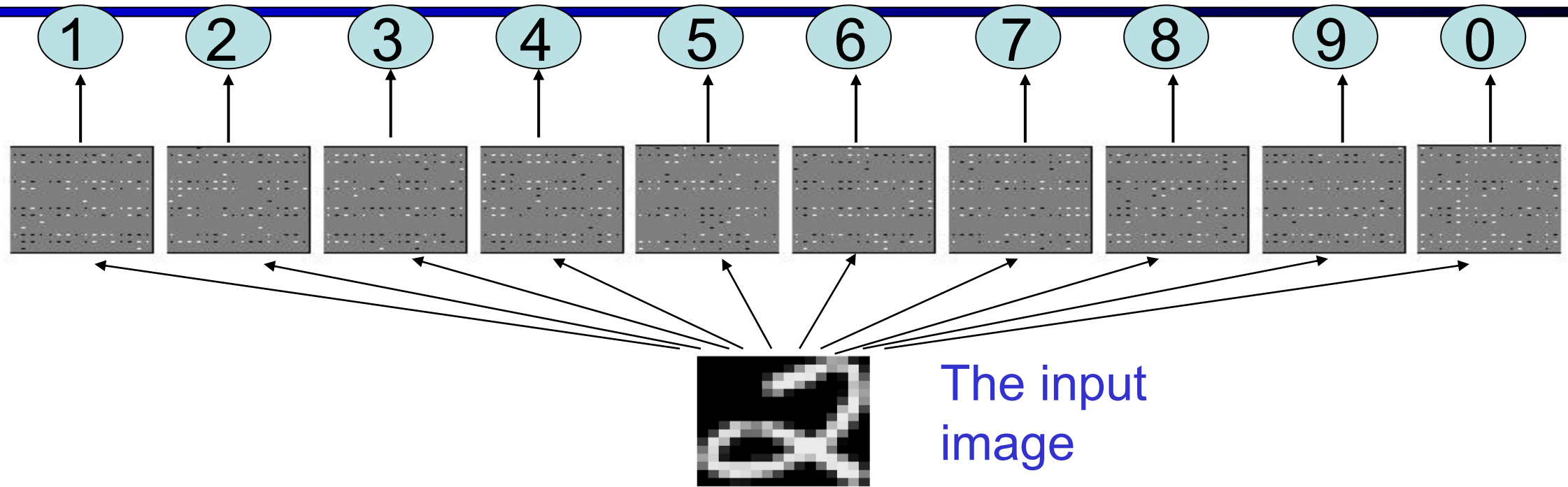
9 9 9 9 9 9 9 9 9 9

A very simple way to recognize handwritten shapes

- Consider a neural network with two layers of neurons.
 - neurons in the top layer represent known shapes.
 - neurons in the bottom layer represent pixel intensities.
- A pixel gets to vote if it has ink on it.
 - Each inked pixel can vote for several different shapes.
- The shape that gets the most votes wins.



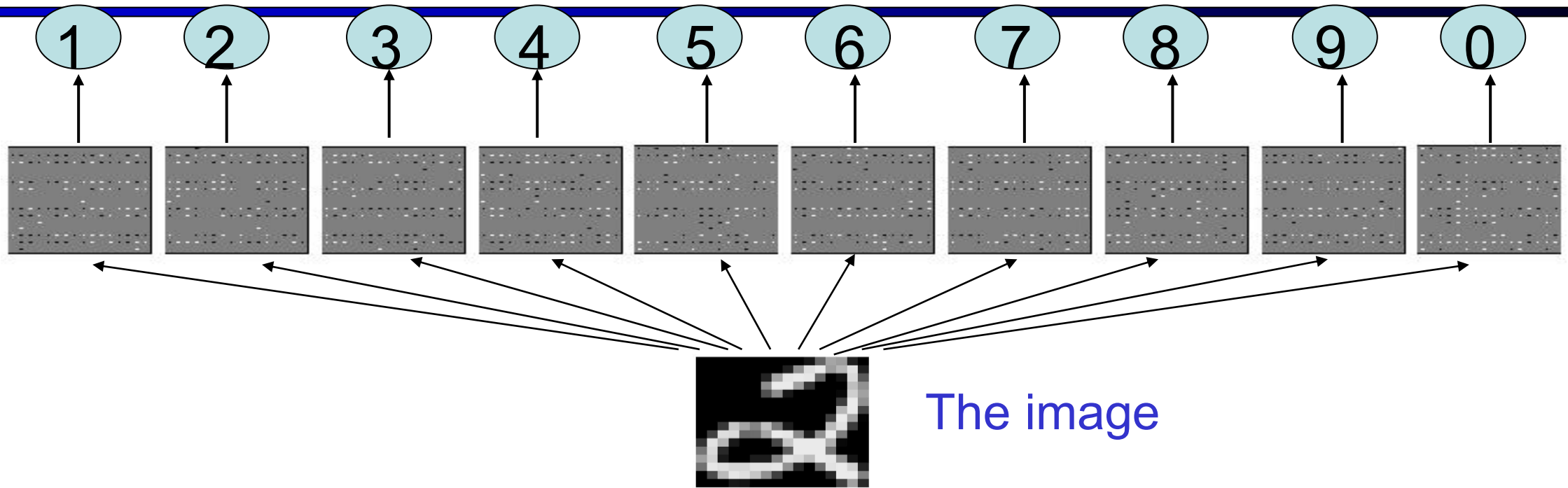
How to display the weights



Give each output unit its own “map” of the input image and display the weight coming from each pixel in the location of that pixel in the map.

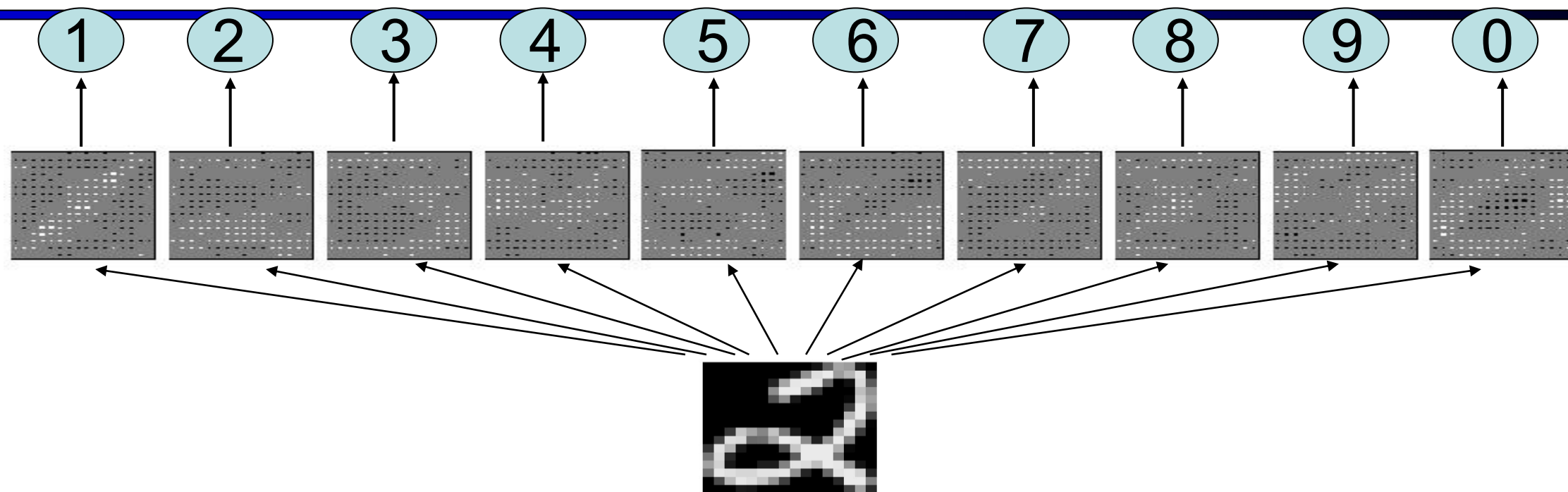
Use a black or white blob with the area representing the magnitude of the weight and the color representing the sign.

How to learn the weights

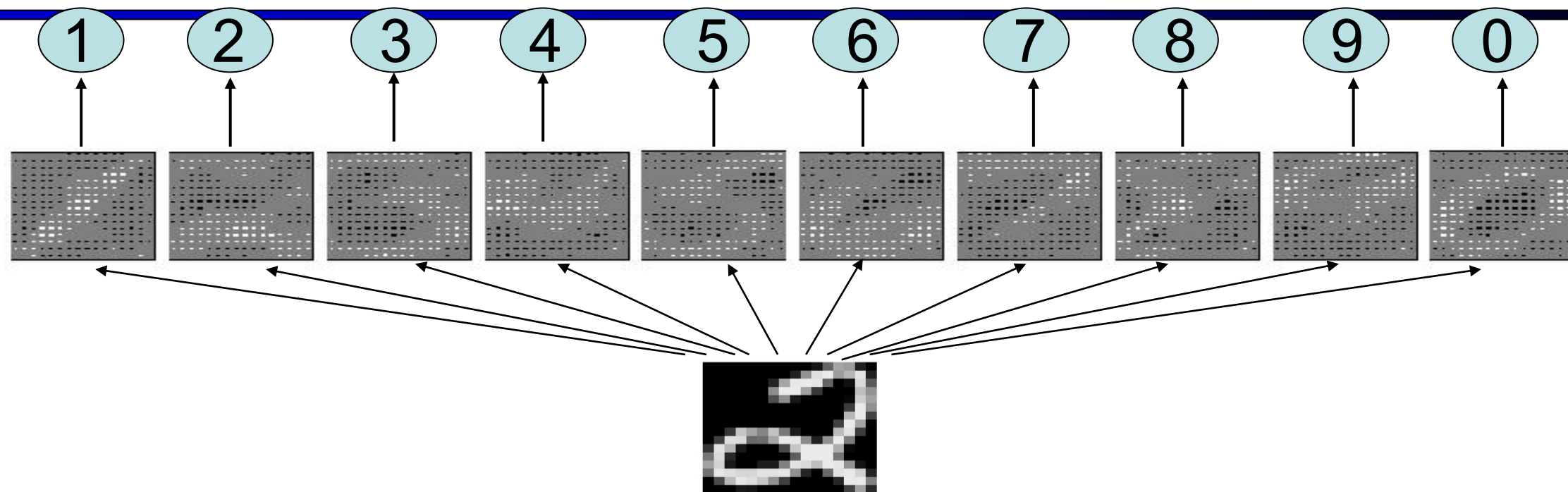


Show the network an image and **increment** the weights from active pixels to the correct class.

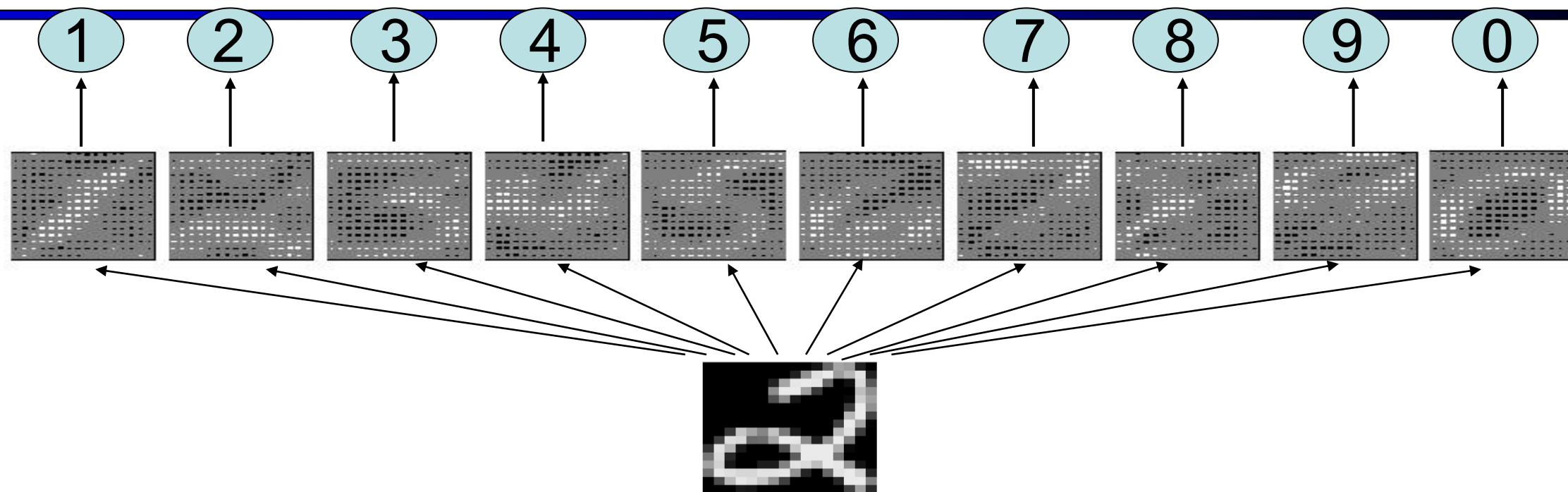
Then **decrement** the weights from active pixels to whatever class the network guesses.



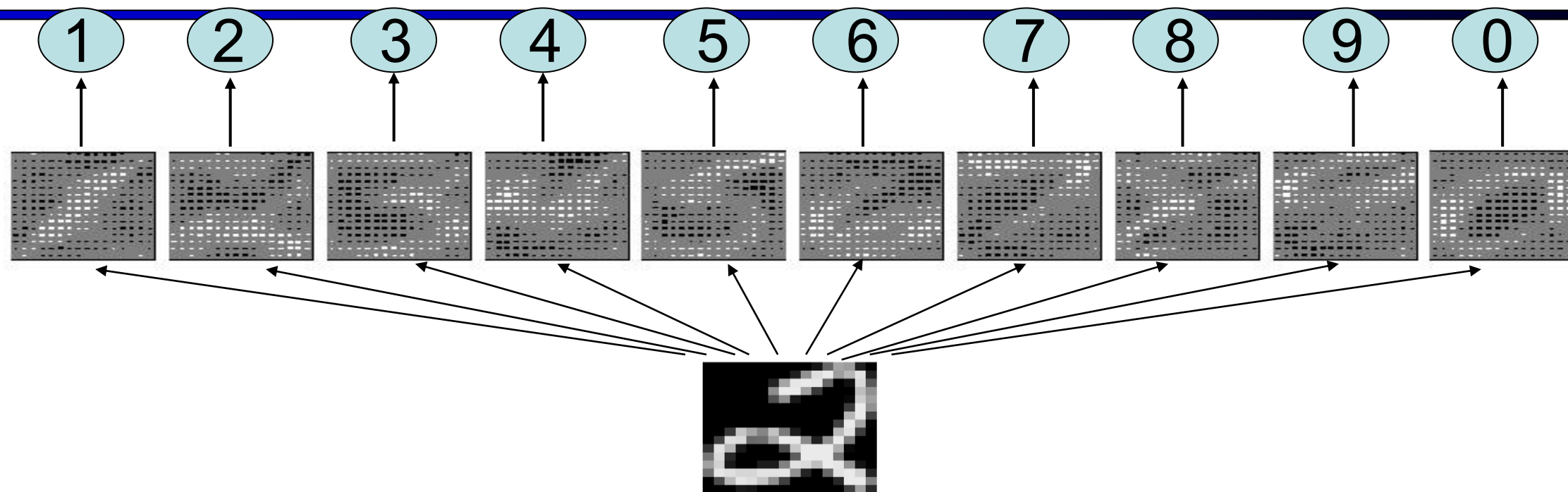
The image



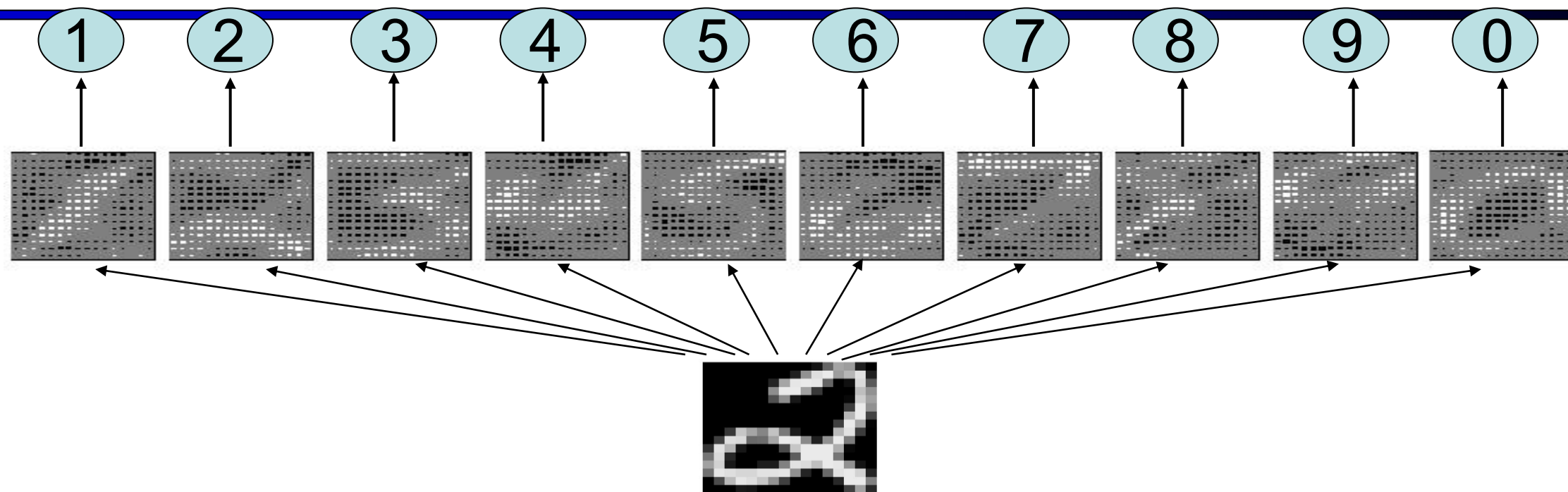
The image



The image

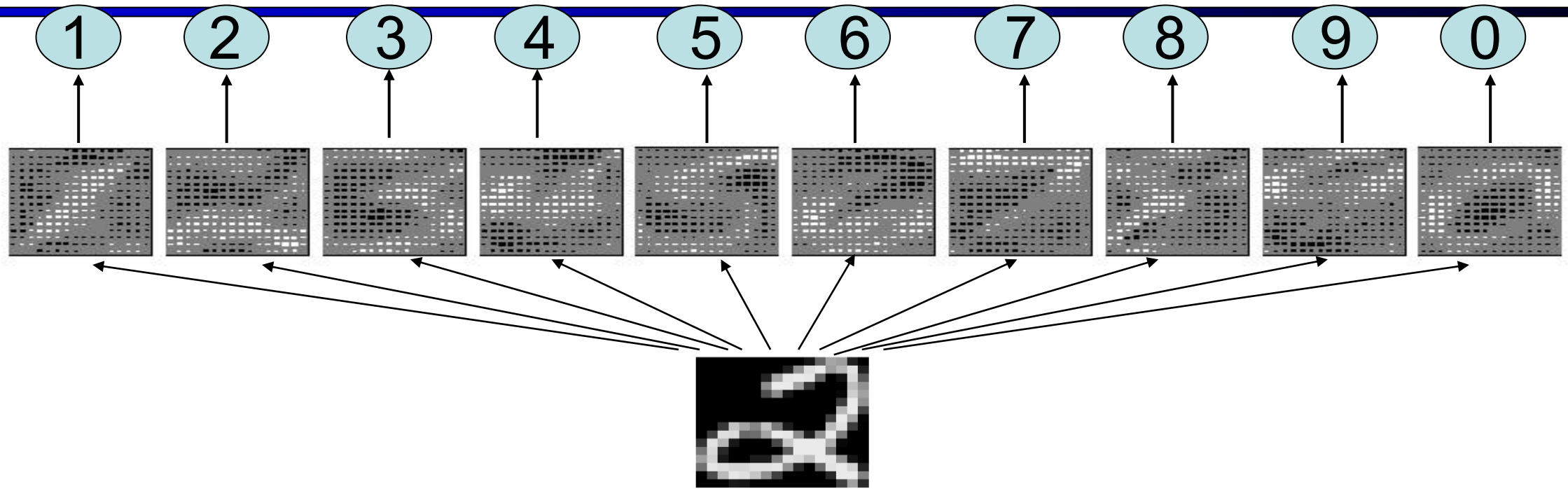


The image



The image

The learned weights



The image

The details of the learning algorithm will be explained in the future.

The limitations of Perceptrons

- If you are allowed to choose the features by hand and if you use enough features, you can do almost anything.
 - For binary input vectors, we can have a separate feature unit for each of the exponentially many binary vectors and so we can make any possible discrimination on binary input vectors.
 - This type of table look-up won't generalize.
- But once the hand-coded features have been determined, there are very strong limitations on what a perceptron can learn.

Core Mathematics of MLP Training

Goal: Minimize loss by adjusting all weights using the error gradient.

Process: Forward Pass → Backward Pass (Backpropagation) → Weight Update

1. Forward Pass

For each layer l :

$$\mathbf{Z}^{[l]} = \mathbf{W}^{[l]} \mathbf{A}^{[l-1]} + \mathbf{b}^{[l]}$$

$$\mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]})$$

Where:

- $\mathbf{A}^{[0]} = \mathbf{X}$ (input data)
- $g^{[l]}$ is activation function (Sigmoid, ReLU, etc.)

Final Loss: $L = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\mathbf{A}_i^{[L]}, \mathbf{Y}_i)$

Core Mathematics of MLP Training

Goal: Minimize loss by adjusting all weights using the error gradient.

Process: Forward Pass → Backward Pass (Backpropagation) → Weight Update

2. Backward Pass (Backpropagation)

We calculate how much each weight contributes to the final error using the chain rule, working backwards from output to input.

Output Layer ($l = L$):

$$d\mathbf{Z}^{[L]} = \frac{\partial L}{\partial \mathbf{Z}^{[L]}} = (\mathbf{A}^{[L]} - \mathbf{Y}) \odot g'^{[L]}(\mathbf{Z}^{[L]})$$

$$d\mathbf{W}^{[L]} = \frac{1}{m} d\mathbf{Z}^{[L]} \mathbf{A}^{[L-1]T}$$

$$d\mathbf{b}^{[L]} = \frac{1}{m} \sum_{i=1}^m d\mathbf{Z}_i^{[L]}$$

Hidden Layers ($l = L - 1, L - 2, \dots, 1$):

$$d\mathbf{Z}^{[l]} = (\mathbf{W}^{[l+1]T} d\mathbf{Z}^{[l+1]}) \odot g'^{[l]}(\mathbf{Z}^{[l]})$$

Key Insight: The error $d\mathbf{Z}^{[l+1]}$ from layer $l + 1$ is propagated backward to calculate $d\mathbf{Z}^{[l]}$ for layer l .

$$d\mathbf{W}^{[l]} = \frac{1}{m} d\mathbf{Z}^{[l]} \mathbf{A}^{[l-1]T}$$

$$d\mathbf{b}^{[l]} = \frac{1}{m} \sum_{i=1}^m d\mathbf{Z}_i^{[l]}$$

Core Mathematics of MLP Training

Goal: Minimize loss by adjusting all weights using the error gradient.

Process: Forward Pass → Backward Pass (Backpropagation) → Weight Update

3. Weight Update

For all layers $l = 1$ to L :

$$\mathbf{W}^{[l]} = \mathbf{W}^{[l]} - \alpha d\mathbf{W}^{[l]}$$

$$\mathbf{b}^{[l]} = \mathbf{b}^{[l]} - \alpha d\mathbf{b}^{[l]}$$

Where α is the learning rate.

Core Mathematics of MLP Training

Goal: Minimize loss by adjusting all weights using the error gradient.

Process: Forward Pass → Backward Pass (Backpropagation) → Weight Update

Simple Example: 2-Layer MLP

Network: Input → Hidden Layer (2 neurons) → Output Layer (1 neuron)

Activation: Sigmoid for both layers

Data: Single sample $x = 1.0$, target $y = 0.5$

Forward Pass:

1. Hidden Layer: $z_1 = w_1x + b_1$, $a_1 = \sigma(z_1)$

2. Output Layer: $z_2 = w_2a_1 + b_2$, $a_2 = \sigma(z_2)$

3. Loss: $L = \frac{1}{2}(a_2 - y)^2$

Backward Pass:

1. Output Layer:

$$\frac{\partial L}{\partial z_2} = (a_2 - y) \cdot \sigma'(z_2)$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial z_2} \cdot a_1$$

2. Hidden Layer:

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial z_2} \cdot w_2 \cdot \sigma'(z_1)$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_1} \cdot x$$

Weight Update:

$$w_1 = w_1 - \alpha \frac{\partial L}{\partial w_1}$$

$$w_2 = w_2 - \alpha \frac{\partial L}{\partial w_2}$$

Extreme Learning Machines

Extreme Learning Machine (ELM)

- It is a specific type of single-hidden-layer feedforward neural network (SLFN). Its key innovation is that it **does not use backpropagation**.
- Instead of iteratively tuning all weights, ELM works in two simple steps:
 - 1. Random Projection:** The weights between the input and hidden layer are set **randomly** and **never updated**.
 - 2. Linear Solution:** The weights between the hidden layer and output layer are calculated **analytically** using a least-squares method.

The ELM Algorithm in 3 Steps

■ Step 1: Randomly Initialize Hidden Layer

- Generate input weights $\mathbf{W}$ and biases $\mathbf{b}$ randomly from any continuous probability distribution (e.g., Gaussian, Uniform) and freeze them.

■ Step 2: Calculate Hidden Layer Output Matrix ($\mathbf{H}$)

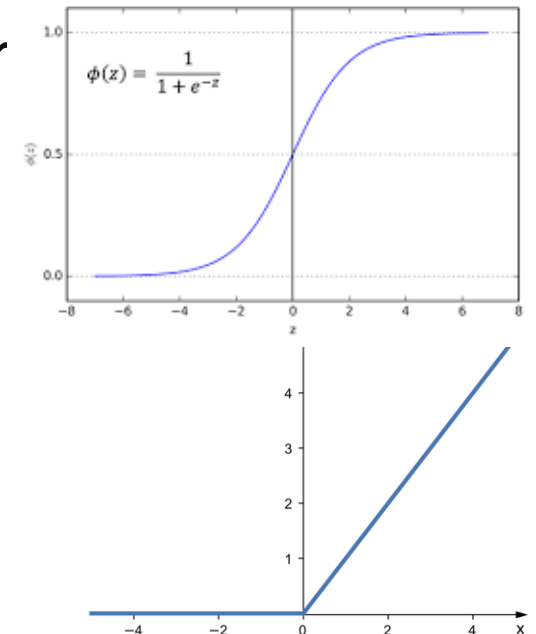
- For a given dataset with N samples, pass all inputs through the hidden layer.
- The output of the hidden layer for the i -th sample is a vector

$$h(x_i) = g(Wx_i + b)$$

where g is the activation function (e.g., sigmoid, ReLU).

- Stack these vectors to form the **Hidden Layer Output Matrix $\mathbf{H}$** , which has dimensions $N \times L$

(number of samples $\times$ number of hidden neurons).



The ELM Algorithm in 3 Steps

- **Step 3: Analytically Solve for Output Weights (β)**

- The output of the network is $\mathbf{T}=\mathbf{H}\beta$, where $\mathbf{T}$ is the target matrix.
- Instead of using gradient descent, we find β by solving the linear system:

$$\beta=\mathbf{H}^{\dagger}\mathbf{T}$$

where $\mathbf{H}^{\dagger}$ is the **Moore-Penrose pseudo-inverse** of $\mathbf{H}$.

- This gives the optimal output weights (in the least-squares sense) in one step.

Comparison: ELM vs. Traditional MLP

Feature	Traditional MLP (with Backprop)	Extreme Learning Machine (ELM)
Training	Iterative (Gradient Descent)	One-Shot (Analytical)
Speed	Slow	Extremely Fast
Input-Hidden Weights	Learned via Backprop	Randomly Fixed
Output Weights	Learned via Backprop	Calculated by Pseudo-inverse
Risk	Local Minima, Overfitting	No Local Minima (Convex problem once H is fixed)

Scenario 1: The General Case (Using Singular Value Decomposition - SVD)

- This is the most robust and widely applicable method. It works for any matrix, including those that are rank-deficient, non-square, or ill-conditioned.

Let $\mathbf{H}$ be an $m \times n$ matrix.

Step-by-Step Explanation:

1. Perform the Singular Value Decomposition (SVD):

Decompose the matrix $\mathbf{H}$ into three component matrices:

$$\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$$

- $\mathbf{U}$ is an $m \times m$ complex unitary matrix (its columns are the left-singular vectors of $\mathbf{H}$).
- $\mathbf{\Sigma}$ is an $m \times n$ rectangular diagonal matrix. Its diagonal elements $\sigma_1, \sigma_2, \dots, \sigma_r$ are the **singular values** of $\mathbf{H}$, which are non-negative real numbers. All other elements are zero. r is the rank of $\mathbf{H}$.
- $\mathbf{V}^*$ is the conjugate transpose of an $n \times n$ complex unitary matrix $\mathbf{V}$ (the columns of $\mathbf{V}$ are the right-singular vectors of $\mathbf{H}$).

Scenario 1: The General Case (Using Singular Value Decomposition - SVD)

2. Compute the Pseudo-inverse of the Σ Matrix:

- Take the transpose of Σ to get an $n \times m$ matrix.
- **Invert the non-zero singular values.** Replace every *non-zero* singular value σ_i on the diagonal with its reciprocal $1/\sigma_i$.
- **Crucial Step:** For numerical stability, define a small tolerance value (e.g., $\text{tol} = 1e - 10$). Any singular value less than this tolerance is treated as zero and is *not* inverted (its reciprocal is set to 0). This prevents division by extremely small numbers that amplify rounding errors.

The resulting matrix is Σ^+ , an $n \times m$ matrix.

3. Compute the Moore-Penrose Pseudo-inverse $\mathbf{H}^+$:

Calculate the product of the three component matrices, in reverse order and using their conjugate transposes:

$$\mathbf{H}^+ = \mathbf{V}\Sigma^+\mathbf{U}^*$$

This formula arises because for unitary matrices, the conjugate transpose $\mathbf{U}^*$ is equal to the inverse $\mathbf{U}^{-1}$. The result, $\mathbf{H}^+$, is an $n \times m$ matrix.

SVD Decomposition

The goal of SVD is to decompose any $m \times n$ matrix A into three special matrices:

$$A = U\Sigma V^T$$

where:

- U is an $m \times m$ orthogonal matrix ($U^T U = I$). Its columns are called the **left-singular vectors**.
- Σ (Sigma) is an $m \times n$ "diagonal" matrix. Its only non-zero elements are on the main diagonal: $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$, where r is the rank of A . These are the **singular values**.
- V^T is the transpose of an $n \times n$ orthogonal matrix V ($V^T V = I$). The columns of V are called the **right-singular vectors**.

Geometric Intuition: SVD states that any linear transformation (A) can be broken down into three simple steps (performed by V^T , Σ , and U):

1. A rotation/reflection in the domain (V^T).
2. A scaling along the perpendicular axes (Σ).
3. A rotation/reflection in the codomain (U).

SVD Decomposition

Step-by-Step Derivation:

1. Form the Covariance Matrices:

Compute AA^T (an $m \times m$ matrix) and $A^T A$ (an $n \times n$ matrix). Both of these are symmetric and positive semi-definite.

2. Find the Eigenpairs of AA^T and $A^T A$:

- The eigenvectors of AA^T are the **left-singular vectors** (U) of A .
- The eigenvectors of $A^T A$ are the **right-singular vectors** (V) of A .
- **Crucially:** The non-zero eigenvalues of both AA^T and $A^T A$ are identical and are equal to the **squares of the singular values** of A . That is, if λ is an eigenvalue of AA^T (or $A^T A$), then $\sigma = \sqrt{\lambda}$ is a singular value of A .

SVD Decomposition

3. Assemble the Matrices:

- Construct U : Place the orthonormal eigenvectors of AA^T as its columns. It's standard to order them by decreasing eigenvalue.
- Construct V : Place the orthonormal eigenvectors of $A^T A$ as its columns, ordered similarly to U .
- Construct Σ : Create an $m \times n$ zero matrix. On its main diagonal (i, i positions), place the singular values $\sigma_i = \sqrt{\lambda_i}$ in descending order.

This derivation is perfect for understanding the theory but is **not the way it's done numerically** because forming AA^T and $A^T A$ can introduce numerical instability.

SVD Decomposition

- Simpler way for V Calculation:
- We remember that:
 - $A = U\Sigma V^*$
- Multiply both sides in U^* and getting transpose:
 - $U^*A = U^*U\Sigma V^* = \Sigma V^*$
 - $A^*U = V\Sigma$
- Thus for any column i
 - $A^*u_i = \sigma_{i,i}v_i$
 - $v_i = \frac{1}{\sigma_{i,i}}A^*u_i$

Radial Basis Functions Network

Radial Basis Functions Network

Radial Basis Functions (RBFs) are mathematical functions whose value depends only on the distance from a **center point**.

Core Concept:

- **"Radial"** = symmetric around a center
- **"Basis"** = building blocks for more complex functions
- They create a "bump" or "hill" at their center that decays with distance

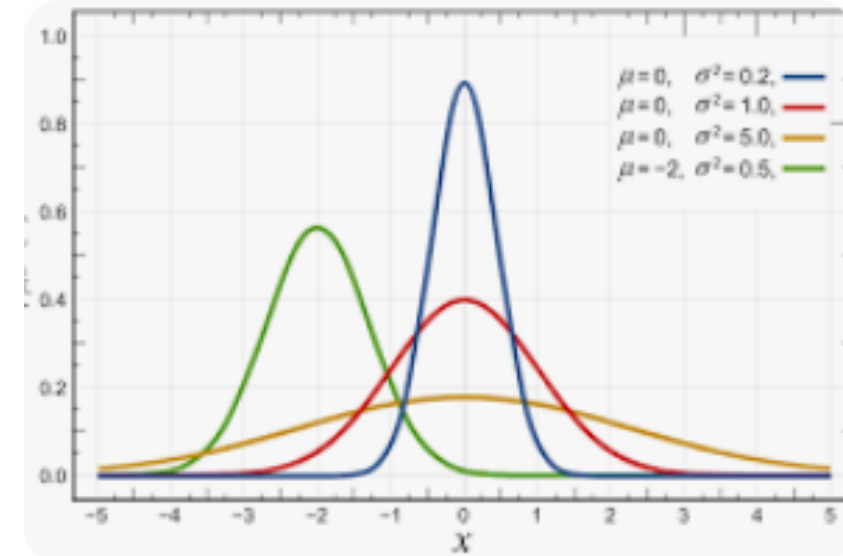
Common RBF Types:

1. **Gaussian:** $\phi(r) = e^{-(\epsilon r)^2}$
 - Most popular, smooth, infinitely differentiable
 - ϵ controls the width ("spread")
2. **Multiquadric:** $\phi(r) = \sqrt{1 + (\epsilon r)^2}$
 - Grows with distance
3. **Inverse Multiquadric:** $\phi(r) = \frac{1}{\sqrt{1 + (\epsilon r)^2}}$
 - Decays with distance
4. **Thin Plate Spline:** $\phi(r) = r^2 \log(r)$
 - Common in interpolation

Where $r = \|x - c\|$ = distance from input x to center c .

RBF Network

- **Core Concept: Localized Function Approximation**
- An **RBF Network** is a type of neural network that uses **distance** and **localized functions** to model data. It's structurally similar to an MLP but works on a fundamentally different principle.
- **Key Idea:** Instead of using weighted sums and sigmoid/ReLU activations (which have global influence), RBF networks use **Radial Basis Functions** (like Gaussian bells) that have **local influence**. Each neuron is sensitive to a specific region of the input space.



RBF Network

- **The 3-Layer Architecture**

- **1. Input Layer**

Receives the input vector.

- **2. Hidden (RBF) Layer**

Each neuron represents a **prototype** or **cluster center** in the input space.

Its activation is determined by a **Radial Basis Function**, which measures the *similarity* between the input and the neuron's center.

The most common RBF is the **Gaussian Function**:

$$\phi_j(\mathbf{x}) = \exp \left(-\frac{\|\mathbf{x} - \mathbf{c}_j\|^2}{2\sigma_j^2} \right)$$

- $\mathbf{x}$ is the input vector.
- $\mathbf{c}_j$ is the center of the j -th RBF neuron.
- σ_j is the width (spread) of the j -th neuron.
- $\|\mathbf{x} - \mathbf{c}_j\|$ is the Euclidean distance.

RBF Network

- **Interpretation:** The output is 1 if the input is exactly at the center and decays to 0 as the input moves away. It creates a "bump" in the input space.

■ 3. Output Layer

Produces the final output as a **linear combination** of the hidden layer activations. Then for $k = 1, \dots, n$:

$$y_k(\mathbf{x}) = \sum_{j=1}^M w_{kj} \phi_j(\mathbf{x}) + b_k$$

Where w_{kj} are the weights and b_k are the biases.

RBF Network

■ **3. Output Layer:** For $k = 1, \dots, n$:

$$y_k(\mathbf{x}) = \sum_{j=1}^M w_{kj} \phi_j(\mathbf{x}) + b_k$$

Where w_{kj} are the weights and b_k are the biases. Equivalently, substitute b_k as the zero-column of vector w_j , and add 1 as the zero-column of $\Phi_j(x)$, we have: $y_k(x) = \sum_{j=0}^M w_{kj} \Phi_j(x)$.

Thus, for all of the training samples $x = 1, \dots, K$, we have: $y(x) = W\Phi(x)$ where $y(x)$, and b are n-vectors and $\Phi(x)$ is m-vector. In matrix form, we can write:

$$Y = W\Phi$$

where Y is target matrix whose columns are $[y(x_1), \dots, y(x_K)]$ and Φ is matrix of hidden nodes' output, whose columns are $[\Phi(x_1), \dots, \Phi(x_K)]$.

Training an RBF Network (2-Stage Process)

- Training is typically much faster than for an MLP because it's often **non-iterative**.
- **Stage 1: Unsupervised Center Selection (c_j)**
 - **Goal:** Determine the centers c_j of the RBF neurons.
 - **Methods:**
 - **K-Means Clustering:** The most common method. The cluster centroids found by K-Means become the centers c_j .
 - **Random Selection:** Randomly choose M data points as centers.

Training an RBF Network (2-Stage Process)

- **Stage 2: Supervised Width Calculation (σ_j) and Output Weight Learning (w_{kj})**
- **Width (σ_j):** Often set heuristically. A common method is to set it as the average distance to the P nearest centers

$$\sigma_j = \frac{1}{P} \sum_{p=1}^P \|\mathbf{c}_j - \mathbf{c}_p\|$$

Training an RBF Network (2-Stage Process)

- **Output Weights (w_{kj}):** Once the centers and widths are fixed, hidden vectors $\phi_j(x)$ can be computed for all training samples x and matrix Φ can be considered. The problem of finding the output weights becomes a **linear regression** problem:

$$Y = W\Phi$$

- If $\Phi\Phi^t$ is invertible, it can be simplified as $Y\Phi^t = W\Phi\Phi^t$ or:

$$W = Y\Phi^t(\Phi\Phi^t)^{-1}$$

- Otherwise, using SVD decomposition, we can determine pseudo-inverse **analytically** and give $W = Y\Phi^+$.

Comparison: RBF Network vs. MLP

Feature	Multilayer Perceptron (MLP)	Radial Basis Function (RBF) Network
Activation	Global (e.g., Sigmoid, ReLU)	Local (e.g., Gaussian)
Hidden Layer	Distributes representation across all neurons	Each neuron represents a local region
Training	Iterative (Backpropagation)	Often 2-Stage & Fast (Clustering + Linear Regression)
Interpretability	Low (Black Box)	Higher (Neurons correspond to data clusters)
Function Approximation	Global	Local

Self-Organizing Map (SOM)

Self-Organizing Map (SOM)

- **Core Concept: An "Intelligent" Map**
- A **Self-Organizing Map (SOM)** or **Kohonen Map** is an unsupervised learning algorithm that creates a **low-dimensional** (typically 2D), discrete representation of high-dimensional data.
- Think of it as an **automatic data clustering and visualization tool**.
- **Key Idea:** It preserves the **topological properties** of the input space. This means similar data points in the high-dimensional space are mapped to **nearby nodes** on the 2D map.

Self-Organizing Map (SOM)

- **The SOM Architecture**
 - **Input Layer:** Receives the high-dimensional data vectors.
 - **Output/Map Layer:** A grid (usually 1D or 2D) of neurons (or nodes). Each node has:
 - A **position** in the grid (e.g., coordinates (2,3)).
 - A **weight vector** of the same dimension as the input data. This is the node's "model" of the data.
- There are no activation functions or weighted sums in the traditional neural network sense.

The Training Algorithm (The "Self-Organizing" Process)

- Training is competitive and cooperative, inspired by neural development in the brain.
- Step 1: Initialization
 - Initialize the weight vectors for all nodes in the map (randomly or via sampling).
- Step 2: Competition
 - For a randomly chosen input data vector x_i , find the Best Matching Unit (BMU).
- The BMU is the node whose weight vector w is closest to x (e.g., using Euclidean distance). ($m_{BMU} = \operatorname{argmin}_{m_j} ||x_i - w(m_j)||$.)

The Training Algorithm (The "Self-Organizing" Process)

■ Step 3: Cooperation

- The BMU acts as an anchor point. We identify its neighbors on the map grid.
- The key is that **not all neighbors are updated equally**. The influence of the BMU is defined by a **Neighborhood Function $h(t)$** .
- This function is often a Gaussian centered on the BMU. It means:
 - Nodes **close to the BMU** are strongly influenced.
 - Nodes far away are weakly influenced.
 - The neighborhood radius shrinks over time during training.
- Definition of $h(t)$ for any neuron position j :

$$h_j = \exp\left(-\frac{GridDistance^2}{2\sigma(t)^2}\right) = \exp\left(-\frac{\|m_j - m_{BMU}\|^2}{2\sigma(t)^2}\right)$$

The Training Algorithm (The "Self-Organizing" Process)

- Step 4: Adaptation (Learning)
 - The BMU and its neighbors update their weight vectors to become more like the input vector x .
- The update rule for a node j is:
 - $w_j(new) = w_j(old) + \eta(t) * h_j * (x - w_j(old))$
- Where:
 - $\eta(t)$ is the learning rate, which decreases over time.
 - h_j is the neighborhood function, which also decreases over time.
- Step 5: Iteration
- Repeat Steps 2-4 for many iterations, gradually decreasing the learning rate and neighborhood radius.

Example: Consider 5 samples with 4 Dimensions

1. Problem Setup

Data samples (5 samples, 4 features each):

$$X = \begin{bmatrix} 2.0 & 1.5 & 0.0 & 0.5 \\ 1.0 & 2.0 & 1.0 & 1.0 \\ 0.0 & 0.5 & 2.0 & 1.5 \\ 1.5 & 0.0 & 1.0 & 2.0 \\ 0.5 & 1.0 & 1.5 & 0.0 \end{bmatrix}$$

Each row is a sample: $x^{(1)}, x^{(2)}, \dots, x^{(5)}$.

Example: Consider 5 samples with 4 Dimensions

SOM grid:

A 2×2 map (4 neurons).

Neuron positions: $m_{11}, m_{12}, m_{21}, m_{22}$.

Initialization:

Initialize each neuron's weight vector randomly (dimension = 4).

Let's choose initial weights (small random values, but structured for clarity):

$$W_{\text{initial}} = \begin{bmatrix} m_{11} : & [1.0, 1.0, 1.0, 1.0] \\ m_{12} : & [1.5, 0.5, 0.5, 1.5] \\ m_{21} : & [0.5, 1.5, 1.5, 0.5] \\ m_{22} : & [0.0, 0.0, 2.0, 2.0] \end{bmatrix}$$

Example: Consider 5 samples with 4 Dimensions

Grid coordinates:

We assign (row, col) coordinates to neurons for neighborhood calculation:

- m_{11} at (0,0)
- m_{12} at (0,1)
- m_{21} at (1,0)
- m_{22} at (1,1)

Example: Consider 5 samples with 4 Dimensions

- **Training Parameters (for this example)**

For any training sample x_i and any neuron position m_j , determine:

$$d_j = \| x_i - w(m_j) \|$$

It is the **grid distance** (not weight distance).

Iteration 1

Example: Consider 5 samples with 4 Dimensions

Iteration 1, Step 2: Competition

Pick randomly $x = x^{(1)} = [2.0, 1.5, 0.0, 0.5]$.

Compute Euclidean distance from x to each neuron's weight vector:

$$\text{For } m_{11} \quad d = \|x_1 - w(m_{11})\| = \sqrt{(2.0-1.0)^2 + (1.5-1.0)^2 + (0.0-1.0)^2 + (0.5-1.0)^2} = \sqrt{2.5} \approx 1.581$$

$$\text{For } m_{12} \quad d \approx 1.581$$

$$\text{For } m_{21} \quad d \approx 2.121$$

$$\text{For } m_{22} \quad d \approx 3.536$$

$m_{BMU} = \operatorname{argmin}_{m_j} \|x_1 - w(m_j)\|$: Both m_{11} and m_{12} tie at ≈ 1.581 . Let's pick m_{11} as m_{BMU} .

Example: Consider 5 samples with 4 Dimensions

- Neighborhood function: Gaussian

$$h_j = \exp\left(-\frac{\text{GridDistance}^2}{2\sigma(t)^2}\right) = \exp\left(-\frac{\|m_j - m_{BMU}\|^2}{2\sigma(t)^2}\right)$$

$\|m_j - m_{BMU}\|$ are:

1. To m_{11} (BMU itself):

$$\|(0, 0) - (0, 0)\| = 0$$

2. To m_{12} :

$$\|(0, 1) - (0, 0)\| = \sqrt{(0 - 0)^2 + (1 - 0)^2} = 1$$

3. To m_{21} :

$$\|(1, 0) - (0, 0)\| = \sqrt{(1 - 0)^2 + (0 - 0)^2} = 1$$

4. To m_{22} :

$$\|(1, 1) - (0, 0)\| = \sqrt{(1 - 0)^2 + (1 - 0)^2} = \sqrt{2} \approx 1.414$$

Example: Consider 5 samples with 4 Dimensions

- Neighborhood function: Gaussian

$$h_j = \exp\left(-\frac{\text{GridDistance}^2}{2\sigma(t)^2}\right) = \exp\left(-\frac{\|m_j - m_{BMU}\|^2}{2\sigma(t)^2}\right)$$

For this example, let's set $\sigma=0.8$ for this step.

For m_{11} (distance = 0):

$$h = \exp\left(-\frac{0^2}{1.28}\right) = e^0 = 1.000$$

For m_{12} (distance = 1):

$$h = \exp\left(-\frac{1^2}{1.28}\right) = e^{-0.78125} \approx 0.458$$

For m_{21} (distance = 1):

$$h = \exp\left(-\frac{1^2}{1.28}\right) \approx 0.458$$

For m_{22} (distance = 1.414):

$$h = \exp\left(-\frac{(1.414)^2}{1.28}\right) = \exp\left(-\frac{2.0}{1.28}\right) = e^{-1.5625} \approx 0.210$$

Example: Consider 5 samples with 4 Dimensions

- **Interpretation of These h-values**
- These numbers represent **how much influence** the current training sample has on each neuron:
- **BMU (m_{11}):** Full influence (1.000) — will move most toward the sample
- **Immediate neighbors (m_{12} , m_{21}):** Moderate influence (0.458) — will move somewhat toward the sample
- **Diagonal neighbor (m_{22}):** Smaller influence (0.210) — will move slightly toward the sample

The Training Algorithm (The "Self-Organizing" Process)

- Step 4: Adaptation (Learning): Learning rate: $\eta=0.1$

- The update rule for a weight j is:

- $w_j(new) = w_j(old) + \eta(t) * h_j * (x - w_j(old))$

Update for m_{11} (BMU, $h=1.000$):

Current input: $x = [2.0, 1.5, 0.0, 0.5]$

Initial weights:

$$w_{11} = [1.0, 1.0, 1.0, 1.0]$$

$$w_{12} = [1.5, 0.5, 0.5, 1.5]$$

$$w_{21} = [0.5, 1.5, 1.5, 0.5]$$

$$w_{22} = [0.0, 0.0, 2.0, 2.0]$$

$$\Delta w = 0.1 \times 1.000 \times ([2.0, 1.5, 0.0, 0.5] - [1.0, 1.0, 1.0, 1.0])$$

$$= 0.1 \times [1.0, 0.5, -1.0, -0.5] = [0.10, 0.05, -0.10, -0.05]$$

$$w_{11}^{new} = [1.10, 1.05, 0.90, 0.95]$$

Update for m_{12} ($h=0.458$):

$$\Delta w = 0.1 \times 0.458 \times ([2.0, 1.5, 0.0, 0.5] - [1.5, 0.5, 0.5, 1.5])$$

$$= 0.0458 \times [0.5, 1.0, -0.5, -1.0] \approx [0.0229, 0.0458, -0.0229, -0.0458]$$

$$w_{12}^{new} \approx [1.5229, 0.5458, 0.4771, 1.4542]$$

The Training Algorithm (The "Self-Organizing" Process)

■ Step 4: Adaptation (Learning)

Current input: $x = [2.0, 1.5, 0.0, 0.5]$

Initial weights:

$$w_{11} = [1.0, 1.0, 1.0, 1.0]$$

$$w_{12} = [1.5, 0.5, 0.5, 1.5]$$

$$w_{21} = [0.5, 1.5, 1.5, 0.5]$$

$$w_{22} = [0.0, 0.0, 2.0, 2.0]$$

Update for m_{21} ($h=0.458$):

$$\Delta w = 0.1 \times 0.458 \times ([2.0, 1.5, 0.0, 0.5] - [0.5, 1.5, 1.5, 0.5])$$

$$= 0.0458 \times [1.5, 0.0, -1.5, 0.0] \approx [0.0687, 0.0000, -0.0687, 0.0000]$$

$$w_{21}^{\text{new}} \approx [0.5687, 1.5000, 1.4313, 0.5000]$$

Update for m_{22} ($h=0.210$):

$$\Delta w = 0.1 \times 0.210 \times ([2.0, 1.5, 0.0, 0.5] - [0.0, 0.0, 2.0, 2.0])$$

$$= 0.0210 \times [2.0, 1.5, -2.0, -1.5] = [0.0420, 0.0315, -0.0420, -0.0315]$$

$$w_{22}^{\text{new}} = [0.0420, 0.0315, 1.9580, 1.9685]$$

The Training Algorithm (The "Self-Organizing" Process)

- Step 4: Adaptation (Learning): Learning rate: $\eta=0.1$

Current input: $x = [2.0, 1.5, 0.0, 0.5]$

Initial weights:

$$w_{11} = [1.0, 1.0, 1.0, 1.0]$$

$$w_{12} = [1.5, 0.5, 0.5, 1.5]$$

$$w_{21} = [0.5, 1.5, 1.5, 0.5]$$

$$w_{22} = [0.0, 0.0, 2.0, 2.0]$$

Final Weights After One Iteration:

$$m_{11} : [1.1000, 1.0500, 0.9000, 0.9500]$$

$$m_{12} : [1.5229, 0.5458, 0.4771, 1.4542]$$

$$m_{21} : [0.5687, 1.5000, 1.4313, 0.5000]$$

$$m_{22} : [0.0420, 0.0315, 1.9580, 1.9685]$$

The Training Algorithm (The "Self-Organizing" Process)

- Step 4: Adaptation (Learning)

Current input: $x = [2.0, 1.5, 0.0, 0.5]$

Initial weights:

$$w_{11} = [1.0, 1.0, 1.0, 1.0]$$

$$w_{12} = [1.5, 0.5, 0.5, 1.5]$$

$$w_{21} = [0.5, 1.5, 1.5, 0.5]$$

$$w_{22} = [0.0, 0.0, 2.0, 2.0]$$

Final Weights After One Iteration:

$$m_{11} : [1.1000, 1.0500, 0.9000, 0.9500]$$

$$m_{12} : [1.5229, 0.5458, 0.4771, 1.4542]$$

$$m_{21} : [0.5687, 1.5000, 1.4313, 0.5000]$$

$$m_{22} : [0.0420, 0.0315, 1.9580, 1.9685]$$

Iteration 2

Example: Consider 5 samples with 4 Dimensions

Iteration 2: Step 1: Pick New Sample Randomly

Let's pick $x^{(3)} = [0.0, 0.5, 2.0, 1.5]$ (third sample from our original dataset).

For m_{11} $d = \|x_3 - w(m_{11})\| = 1.739$

For m_{12}
 $d \approx 2.155$

For m_{21}
 $d \approx 1.627$

For m_{22}
 $d \approx 0.665$

$m_{BMU} = \operatorname{argmin}_{m_j} \|x_1 - w(m_j)\|$: m_{22} with distance ≈ 0.665 (closest to x_3) is considered as m_{BMU} .

Example: Consider 5 samples with 4 Dimensions

- Neighborhood function: Gaussian

$$h_j = \exp\left(-\frac{\text{GridDistance}^2}{2\sigma(t)^2}\right) = \exp\left(-\frac{\|m_j - m_{BMU}\|^2}{2\sigma(t)^2}\right)$$

$\|m_j - m_{BMU}\|^2$ are:

Neuron	Grid Position	Distance to (1,1)	Distance ²
m_{11}	(0,0)	$\sqrt{(1-0)^2 + (1-0)^2} = \sqrt{2} \approx 1.414$	2.0
m_{12}	(0,1)	$\sqrt{(1-0)^2 + (1-1)^2} = 1$	1.0
m_{21}	(1,0)	$\sqrt{(1-1)^2 + (1-0)^2} = 1$	1.0
m_{22}	(1,1)	0	0

Example: Consider 5 samples with 4 Dimensions

- Neighborhood function: Gaussian

$$h_j = \exp\left(-\frac{\text{GridDistance}^2}{2\sigma(t)^2}\right) = \exp\left(-\frac{\|m_j - m_{BMU}\|^2}{2\sigma(t)^2}\right)$$

For this example, let's set $\sigma=0.8$ for this step.

For m_{22} (BMU, distance²=0):

$$h = \exp\left(-\frac{0}{1.28}\right) = 1.000$$

For m_{12} (distance²=1):

$$h = \exp\left(-\frac{1}{1.28}\right) = e^{-0.78125} \approx 0.458$$

For m_{21} (distance²=1):

$$h \approx 0.458$$

For m_{11} (distance²=2):

$$h = \exp\left(-\frac{2}{1.28}\right) = e^{-1.5625} \approx 0.210$$

The Training Algorithm (The "Self-Organizing" Process)

■ Step 4: Adaptation (Learning): Learning rate: $\eta=0.1$

- The update rule for a weight j is: $w_j(new) = w_j(old) + \eta(t) * h_j * (x - w_j(old))$

Update m_{22} ($h=1.000$):

$$\begin{aligned}\Delta w &= 0.1 \times 1.000 \times ([0.0, 0.5, 2.0, 1.5] - [0.0420, 0.0315, 1.9580, 1.9685]) \\ &= 0.1 \times [-0.0420, 0.4685, 0.0420, -0.4685] \\ &= [-0.00420, 0.04685, 0.00420, -0.04685]\end{aligned}$$

$$\begin{aligned}w_{22}^{new} &= [0.0420 - 0.00420, 0.0315 + 0.04685, 1.9580 + 0.00420, 1.9685 - 0.04685] \\ &= [0.0378, 0.07835, 1.9622, 1.92165]\end{aligned}$$

Update m_{12} ($h=0.458$):

$$\begin{aligned}\Delta w &= 0.1 \times 0.458 \times ([0.0, 0.5, 2.0, 1.5] - [1.5229, 0.5458, 0.4771, 1.4542]) \\ &= 0.0458 \times [-1.5229, -0.0458, 1.5229, 0.0458] \\ &\approx [-0.06979, -0.00210, 0.06979, 0.00210]\end{aligned}$$

$$\begin{aligned}w_{12}^{new} &= [1.5229 - 0.06979, 0.5458 - 0.00210, 0.4771 + 0.06979, 1.4542 + 0.00210] \\ &\approx [1.45311, 0.54370, 0.54689, 1.45630]\end{aligned}$$

Update m_{21} ($h=0.458$):

$$\begin{aligned}\Delta w &= 0.1 \times 0.458 \times ([0.0, 0.5, 2.0, 1.5] - [0.5687, 1.5000, 1.4313, 0.5000]) \\ &= 0.0458 \times [-0.5687, -1.0000, 0.5687, 1.0000] \\ &\approx [-0.02607, -0.04580, 0.02607, 0.04580]\end{aligned}$$

$$\begin{aligned}w_{21}^{new} &= [0.5687 - 0.02607, 1.5000 - 0.04580, 1.4313 + 0.02607, 0.5000 + 0.04580] \\ &\approx [0.54263, 1.45420, 1.45737, 0.54580]\end{aligned}$$

Update m_{11} ($h=0.210$):

$$\begin{aligned}\Delta w &= 0.1 \times 0.210 \times ([0.0, 0.5, 2.0, 1.5] - [1.1000, 1.0500, 0.9000, 0.9500]) \\ &= 0.0210 \times [-1.1000, -0.5500, 1.1000, 0.5500] \\ &= [-0.02310, -0.01155, 0.02310, 0.01155]\end{aligned}$$

$$\begin{aligned}w_{11}^{new} &= [1.1000 - 0.02310, 1.0500 - 0.01155, 0.9000 + 0.02310, 0.9500 + 0.01155] \\ &= [1.07690, 1.03845, 0.92310, 0.96155]\end{aligned}$$

The Training Algorithm (The "Self-Organizing" Process)

- Step 4: Adaptation (Learning)

Step 5: End of Iteration 1

$$m_{11} : [1.1000, 1.0500, 0.9000, 0.9500]$$

$$m_{12} : [1.5229, 0.5458, 0.4771, 1.4542]$$

$$m_{21} : [0.5687, 1.5000, 1.4313, 0.5000]$$

$$m_{22} : [0.0420, 0.0315, 1.9580, 1.9685]$$

Step 5: End of Iteration 2

$$m_{11} : [1.07690, 1.03845, 0.92310, 0.96155]$$

$$m_{12} : [1.45311, 0.54370, 0.54689, 1.45630]$$

$$m_{21} : [0.54263, 1.45420, 1.45737, 0.54580]$$

$$m_{22} : [0.03780, 0.07835, 1.96220, 1.92165]$$

Iteration 3

Example: Consider 5 samples with 4 Dimensions

Iteration 3: Step 1: Pick New Sample Randomly

Let's pick $x^{(2)} = [1.0, 2.0, 1.0, 1.0]$ (second sample from our original dataset).

For m_{11} $d = \|x_2 - w(m_{11})\| = 0.9685$

For m_{12}
 $d \approx 1.6552$

For m_{21}
 $d \approx 0.9605$

For m_{22}
 $d \approx 2.5286$

$m_{BMU} = \operatorname{argmin}_{m_j} \|x_1 - w(m_j)\|$: m_{21} with distance ≈ 0.9605 (closest to x_2) is considered as m_{BMU} .

Example: Consider 5 samples with 4 Dimensions

- Neighborhood function: Gaussian

$$h_j = \exp\left(-\frac{\text{GridDistance}^2}{2\sigma(t)^2}\right) = \exp\left(-\frac{\|m_j - m_{BMU}\|^2}{2\sigma(t)^2}\right)$$

$\|m_j - m_{BMU}\|^2$ are:

Neuron	Grid Position	Distance to (1,0)	Distance ²
m_{21}	(1,0)	0	0
m_{11}	(0,0)	$\sqrt{(1-0)^2 + (0-0)^2} = 1$	1.0
m_{22}	(1,1)	$\sqrt{(1-1)^2 + (0-1)^2} = 1$	1.0
m_{12}	(0,1)	$\sqrt{(1-0)^2 + (0-1)^2} = \sqrt{2} \approx 1.414$	2.0

Example: Consider 5 samples with 4 Dimensions

- Neighborhood function: Gaussian

$$h_j = \exp\left(-\frac{\text{GridDistance}^2}{2\sigma(t)^2}\right) = \exp\left(-\frac{\|m_j - m_{BMU}\|^2}{2\sigma(t)^2}\right)$$

For this example, let's set $\sigma=0.8$ for this step.

For m_{21} (BMU, distance²=0):

$$h = 1.000$$

For m_{11} (distance²=1):

$$h = \exp\left(-\frac{1}{1.28}\right) = e^{-0.78125} \approx 0.458$$

For m_{22} (distance²=1):

$$h \approx 0.458$$

For m_{12} (distance²=2):

$$h = \exp\left(-\frac{2}{1.28}\right) = e^{-1.5625} \approx 0.210$$

The Training Algorithm (The "Self-Organizing" Process)

- Step 4: Adaptation (Learning): Learning rate: $\eta=0.1$
 - The update rule for a weight j is: $w_j(new) = w_j(old) + \eta(t) * h_j * (x - w_j(old))$

The Training Algorithm (The "Self-Organizing" Process)

- Step 4: Adaptation (Learning)

Step 5: End of Iteration 2

m_{11} : [1.07690, 1.03845, 0.92310, 0.96155]

m_{12} : [1.45311, 0.54370, 0.54689, 1.45630]

m_{21} : [0.54263, 1.45420, 1.45737, 0.54580]

m_{22} : [0.03780, 0.07835, 1.96220, 1.92165]

Step 5: End of Iteration 3

m_{11} : [1.07338, 1.08249, 0.92662, 0.96331]

m_{12} : [1.44360, 0.57428, 0.55641, 1.44672]

m_{21} : [0.58837, 1.50878, 1.41163, 0.59122]

m_{22} : [0.08187, 0.16636, 1.91813, 1.87944]

Iteration 4

Example: Consider 5 samples with 4 Dimensions

Iteration 4: Step 1: Pick New Sample Randomly

Let's pick $x^{(4)} = [1.5, 0.0, 1.0, 2.0]$ (fourth sample from our original dataset).

For m_{11} $d = \|x_4 - w(m_{11})\| = 1.5601$

For m_{12}
 $d \approx 0.9143$

For m_{21}
 $d \approx 2.2938$

For m_{22}
 $d \approx 1.7018$

$m_{BMU} = \operatorname{argmin}_{m_j} \|x_1 - w(m_j)\|$: m_{12} with distance ≈ 0.9143 (closest to x_4) is considered as m_{BMU} .

Example: Consider 5 samples with 4 Dimensions

- Neighborhood function: Gaussian

$$h_j = \exp\left(-\frac{\text{GridDistance}^2}{2\sigma(t)^2}\right) = \exp\left(-\frac{\|m_j - m_{BMU}\|^2}{2\sigma(t)^2}\right)$$

$\|m_j - m_{BMU}\|^2$ are:

Neuron	Grid Position	Distance to (0,1)	Distance ²
m_{12}	(0,1)	0	0
m_{11}	(0,0)	$\sqrt{(0-0)^2 + (1-0)^2} = 1$	1.0
m_{22}	(1,1)	$\sqrt{(1-0)^2 + (1-1)^2} = 1$	1.0
m_{21}	(1,0)	$\sqrt{(1-0)^2 + (0-1)^2} = \sqrt{2} \approx 1.414$	2.0

The Training Algorithm (The "Self-Organizing" Process)

- Step 4: Adaptation (Learning)

Step 5: End of Iteration 3

m_{11} : [1.07338, 1.08249, 0.92662, 0.96331]

m_{12} : [1.44360, 0.57428, 0.55641, 1.44672]

m_{21} : [0.58837, 1.50878, 1.41163, 0.59122]

m_{22} : [0.08187, 0.16636, 1.91813, 1.87944]

Step 5: End of Iteration 4

m_{11} : [1.09292, 1.03291, 0.92998, 1.01079]

m_{12} : [1.44924, 0.51685, 0.60077, 1.50205]

m_{21} : [0.60751, 1.47710, 1.40299, 0.62080]

m_{22} : [0.14682, 0.15874, 1.87608, 1.88496]

Iteration 5

Example: Consider 5 samples with 4 Dimensions

Iteration 5: Step 1: Pick New Sample Randomly

Let's pick the final sample from our dataset: $x^{(5)} = [0.5, 1.0, 1.5, 0.0]$

For m_{11} $d = \|x_5 - w(m_{11})\| = 1.3036$

For m_{12}
 $d \approx 2.0492$

For m_{21}
 $d \approx 0.7962$

For m_{22}
 $d \approx 2.1275$

$m_{BMU} = \operatorname{argmin}_{m_j} \|x_1 - w(m_j)\|$: m_{21} with distance ≈ 0.7962 (closest to x_5) is considered as m_{BMU} .

Example: Consider 5 samples with 4 Dimensions

- Neighborhood function: Gaussian

$$h_j = \exp\left(-\frac{\text{GridDistance}^2}{2\sigma(t)^2}\right) = \exp\left(-\frac{\|m_j - m_{BMU}\|^2}{2\sigma(t)^2}\right)$$

$\|m_j - m_{BMU}\|^2$ are:

Neuron	Grid Position	Distance to (1,0)	Distance ²
m_{21}	(1,0)	0	0
m_{11}	(0,0)	1	1.0
m_{22}	(1,1)	1	1.0
m_{12}	(0,1)	$\sqrt{2} \approx 1.414$	2.0

The Training Algorithm (The "Self-Organizing" Process)

- Step 4: Adaptation (Learning)

Step 5: End of Iteration 4

$m_{11} : [1.09292, 1.03291, 0.92998, 1.01079]$

$m_{12} : [1.44924, 0.51685, 0.60077, 1.50205]$

$m_{21} : [0.60751, 1.47710, 1.40299, 0.62080]$

$m_{22} : [0.14682, 0.15874, 1.87608, 1.88496]$

Step 5: End of Iteration 5

$m_{11} : [1.06576, 1.03140, 0.95609, 0.96450]$

$m_{12} : [1.42931, 0.52700, 0.61965, 1.47051]$

$m_{21} : [0.59676, 1.42939, 1.41269, 0.55872]$

$m_{22} : [0.16299, 0.19727, 1.85886, 1.79863]$

Observations After 5 Iterations (Complete Pass Through Dataset)

1. Final Specialization Map:

m_{11} : [1.07, 1.03, 0.96, 0.96] (balanced center, all ~ 1.0)

m_{12} : [1.43, 0.53, 0.62, 1.47] (high-low-low-high)

m_{21} : [0.60, 1.43, 1.41, 0.56] (low-high-high-low)

m_{22} : [0.16, 0.20, 1.86, 1.80] (very low-low-very high-high)

Observations After 5 Iterations (Complete Pass Through Dataset)

2. Data Assignment (Which neuron represents which data point best):

Compute distances from each data point to final neurons:

Sample	m_{11}	m_{12}	m_{21}	m_{22}	Best Match
$x^{(1)}$ [2.0,1.5,0.0,0.5]	1.405	1.167	2.523	3.314	m_{12}
$x^{(2)}$ [1.0,2.0,1.0,1.0]	0.982	1.727	0.977	2.758	m_{11}
$x^{(3)}$ [0.0,0.5,2.0,1.5]	1.523	2.326	1.476	0.602	m_{22}
$x^{(4)}$ [1.5,0.0,1.0,2.0]	1.496	0.887	2.250	2.056	m_{12}
$x^{(5)}$ [0.5,1.0,1.5,0.0]	1.300	2.102	0.818	2.127	m_{21}

Observations After 5 Iterations (Complete Pass Through Dataset)

Final Mapping:

- $x^{(1)}$ & $x^{(4)} \rightarrow m_{12}$ (high dim1/dim4, low dim2/dim3)
- $x^{(2)} \rightarrow m_{11}$ (balanced)
- $x^{(3)} \rightarrow m_{22}$ (very low dim1/dim2, very high dim3/dim4)
- $x^{(5)} \rightarrow m_{21}$ (low dim1/dim4, high dim2/dim3)

	High dim1/dim4	Balanced
	Low dim2/dim3	
m12:	Sample 1 & 4 cluster (high1,high4,low2,low3)	Sample 2 cluster (balanced)
m22:	Sample 3 cluster (very low1/2, very high3/4)	Sample 5 cluster (low1/4, high2/3)

Observations After 5 Iterations (Complete Pass Through Dataset)

4. Convergence Analysis:

Comparing Iteration 4 $\rightarrow$ Iteration 5 changes (L2 norm of weight changes):

- m_{11} : $\Delta = 0.051$ (small)
- m_{12} : $\Delta = 0.034$ (small)
- m_{21} : $\Delta = 0.072$ (small)
- m_{22} : $\Delta = 0.087$ (small)

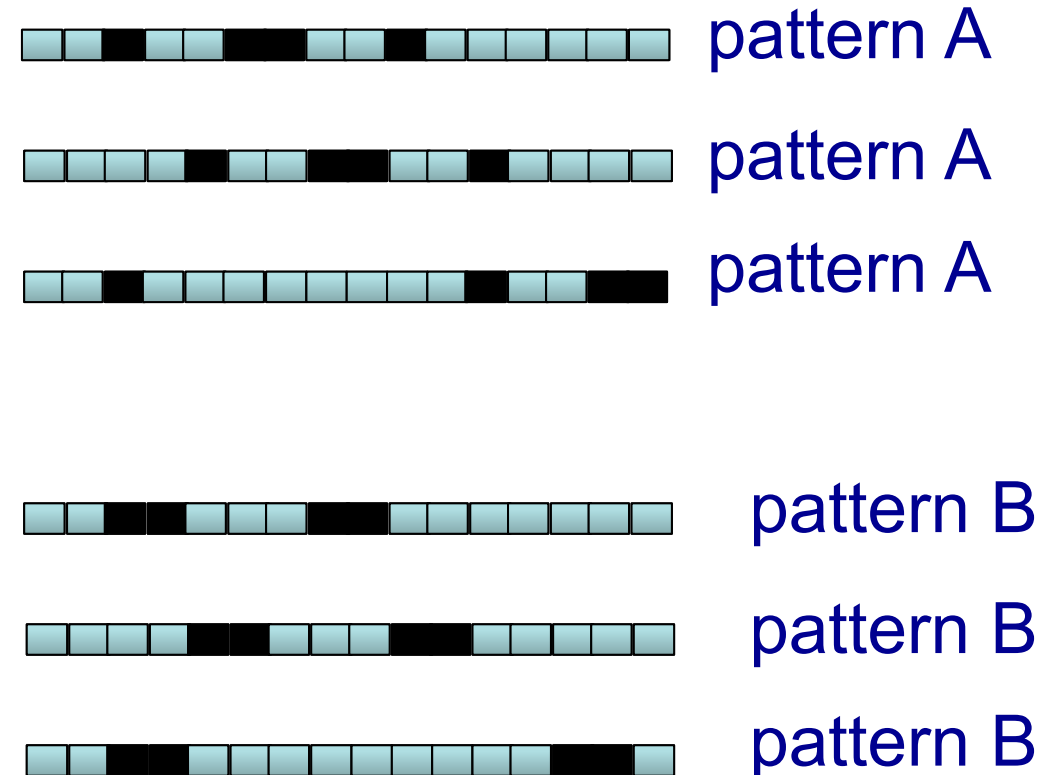
All changes are small (<0.1), suggesting convergence is beginning. In real SOM training, we would:

1. Continue for many more epochs
2. Gradually decrease learning rate η and neighborhood radius σ
3. Eventually the map stabilizes completely

Recurrent Networks

Discriminating simple patterns under translation with wrap-around

- Suppose we just use pixels as the features.
- Can a binary threshold unit discriminate between different patterns that have the same number of on pixels?
 - Not if the patterns can translate with wrap-around!



Sketch of a proof that a binary decision unit cannot discriminate patterns with the same number of on pixels

(assuming translation with wraparound)

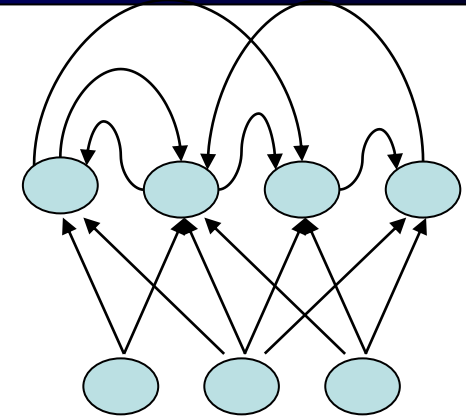
- For pattern A, use training cases in all possible translations.
 - Each pixel will be activated by 4 different translations of pattern A.
 - So the total input received by the decision unit over all these patterns will be four times the sum of all the weights.
- For pattern B, use training cases in all possible translations.
 - Each pixel will be activated by 4 different translations of pattern B.
 - So the total input received by the decision unit over all these patterns will be four times the sum of all the weights.
- But to discriminate correctly, every single case of pattern A must provide more input to the decision unit than every single case of pattern B.
 - This is impossible if the sums over cases are the same.

Why this result is devastating for Perceptrons

- The whole point of pattern recognition is to recognize patterns despite transformations like translation.
- Minsky and Papert's "Group Invariance Theorem" says that the part of a Perceptron that learns cannot learn to do this if the transformations form a group.
 - Translations with wrap-around form a group.
- To deal with such transformations, a Perceptron needs to use multiple feature units to recognize transformations of informative sub-patterns.
 - So the tricky part of pattern recognition must be solved by the hand-coded feature detectors, not the learning procedure.

Recurrent networks

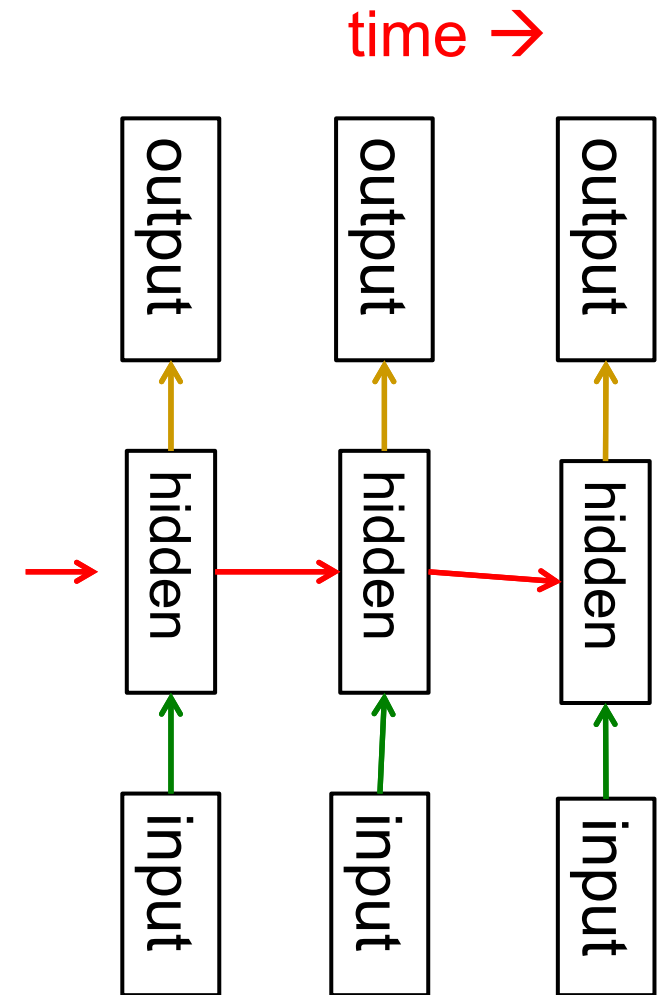
- These have directed cycles in their connection graph.
 - That means you can sometimes get back to where you started by following the arrows.
- They can have complicated dynamics and this can make them very difficult to train.
 - There is a lot of interest at present in finding efficient ways of training recurrent nets.
- They are more biologically realistic.



Recurrent nets with multiple hidden layers are just a special case that has some of the hidden→hidden connections missing.

Recurrent neural networks for modeling sequences

- Recurrent neural networks are a very natural way to model sequential data:
 - They are equivalent to very deep nets with one hidden layer per time slice.
 - Except that they use the same weights at every time slice and they get input at every time slice.
- They have the ability to remember information in their hidden state for a long time.
 - But its very hard to train them to use this potential.



Recurrent Neural Networks (RNNs): Mathematical Foundation

- **Core Concept: Feedback Loops:** Unlike feedforward networks, RNNs contain **directed cycles**, allowing information to persist. This creates an **internal state** or **memory** of previous inputs.

Basic RNN Cell (Elman Network)

For a simple RNN at time step t :

1. **Input:** $\mathbf{x}_t \in \mathbb{R}^n$
2. **Hidden state (memory):** $\mathbf{h}_t \in \mathbb{R}^d$
3. **Output:** $\mathbf{y}_t \in \mathbb{R}^m$

Forward pass equations:

$$\mathbf{h}_t = \sigma(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

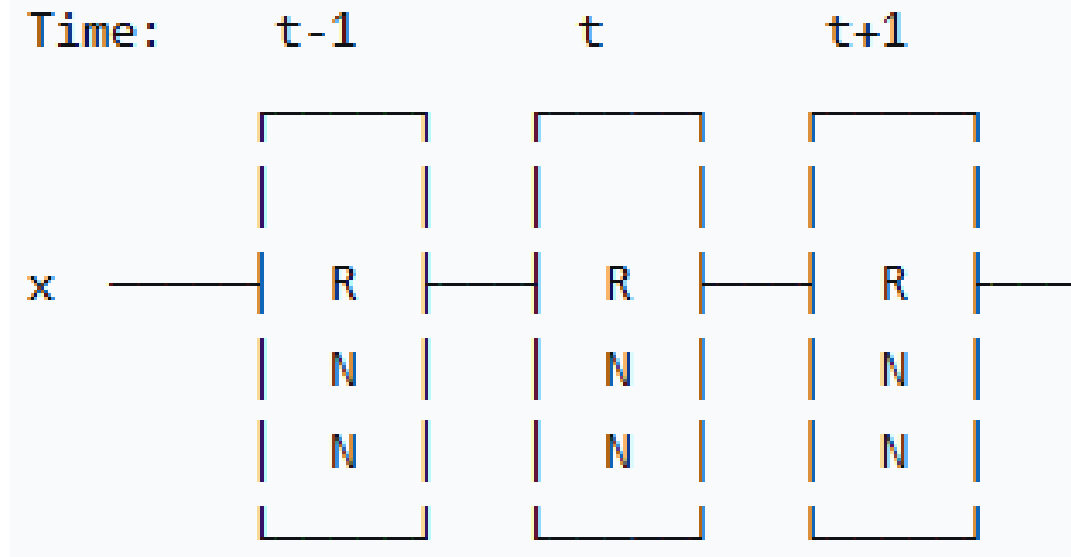
$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$$

Where:

- $\mathbf{W}_{xh}$: input-to-hidden weights
- $\mathbf{W}_{hh}$: hidden-to-hidden (recurrent) weights
- $\mathbf{W}_{hy}$: hidden-to-output weights
- σ : activation function (tanh, ReLU, etc.)
- $\mathbf{h}_{-1}$ is typically initialized to zeros

Recurrent Neural Networks (RNNs): Mathematical Foundation

- **Unfolding Through Time**
- The recurrent connection can be "unfolded" to visualize the network as a deep feedforward network:



- Mathematically, unfolding reveals the **temporal depth**:

$$\mathbf{h}_t = \sigma (\mathbf{W}_{xh} \mathbf{x}_t + \mathbf{W}_{hh} \sigma (\mathbf{W}_{xh} \mathbf{x}_{t-1} + \mathbf{W}_{hh} \sigma (\cdots)))$$

The Training Challenge: Backpropagation Through Time (BPTT)

Gradient Computation

The loss for a sequence of length T :

$$L = \sum_{t=1}^T L_t(\mathbf{y}_t, \hat{\mathbf{y}}_t)$$

To update $\mathbf{W}_{hh}$, we need $\frac{\partial L}{\partial \mathbf{W}_{hh}}$:

$$\frac{\partial L}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial L_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}_{hh}}$$

The critical term is the **Jacobian**:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{i=k+1}^t \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}} = \prod_{i=k+1}^t \mathbf{W}_{hh}^\top \text{diag}(\sigma'(\mathbf{h}_{i-1}))$$

The Training Challenge: Backpropagation Through Time (BPTT)

The Vanishing/Exploding Gradient Problem

Consider the norm of this product:

$$\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| \leq \|\mathbf{W}_{hh}\|^{t-k} \cdot \|\text{diag}(\sigma')\|^{t-k}$$

Since $\|\sigma'\| \leq 1$ for tanh/sigmoid:

- If $\|\mathbf{W}_{hh}\| < 1$: **Vanishing gradients** ($\rightarrow 0$ exponentially)
- If $\|\mathbf{W}_{hh}\| > 1$: **Exploding gradients** ($\rightarrow \infty$ exponentially)

Consequences:

- Vanishing: Can't learn long-term dependencies
- Exploding: Numerical instability, training divergence

The Training Challenge: Backpropagation Through Time (BPTT)

The Vanishing/Exploding Gradient Problem

Consider the norm of this product:

$$\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| \leq \|\mathbf{W}_{hh}\|^{t-k} \cdot \|\text{diag}(\sigma')\|^{t-k}$$

Since $\|\sigma'\| \leq 1$ for tanh/sigmoid:

- If $\|\mathbf{W}_{hh}\| < 1$: **Vanishing gradients** ($\rightarrow 0$ exponentially)
- If $\|\mathbf{W}_{hh}\| > 1$: **Exploding gradients** ($\rightarrow \infty$ exponentially)

Consequences:

- Vanishing: Can't learn long-term dependencies
- Exploding: Numerical instability, training divergence

Practical Training Techniques

1. Gradient Clipping: Threshold gradients to prevent explosion

$$\text{grad} \leftarrow \frac{\text{threshold}}{\|\text{grad}\|} \cdot \text{grad} \quad \text{if } \|\text{grad}\| > \text{threshold}$$

2. Truncated BPTT: Backpropagate only through limited time steps

3. Curriculum Learning: Start with short sequences, gradually increase length

4. Orthogonal Initialization: Initialize W_{hh} as orthogonal matrix to preserve gradient norms

Key Theoretical Results

- 1. Universal Approximation:** RNNs can approximate any measurable sequence-to-sequence mapping (Hammer, 2000)
- 2. Statistical Efficiency:** RNNs can achieve optimal statistical rates for certain sequence learning problems

An example of what recurrent neural nets can now do (to whet your interest!)

- Ilya Sutskever (2011) trained a special type of recurrent neural net to predict the next character in a sequence.
- After training for a long time on a string of half a billion characters from English Wikipedia, he got it to generate new text.
 - It generates by predicting the probability distribution for the next character and then sampling a character from this distribution.
 - The next slide shows an example of the kind of text it generates. Notice how much it knows!

Some text generated one character at a time

by Ilya Sutskever's recurrent neural network

In 1974 Northern Denver had been overshadowed by CNL, and several Irish intelligence agencies in the Mediterranean region. However, on the Victoria, Kings Hebrew stated that Charles decided to escape during an alliance. The mansion house was completed in 1882, the second in its bridge are omitted, while closing is the proton reticulum composed below it aims, such that it is the blurring of appearing on any well-paid type of box printer.

Some text generated one character at a time

by Ilya Sutskever's recurrent neural network

In 1974 Northern Denver had been overshadowed by CNL, and several Irish intelligence agencies in the Mediterranean region. However, on the Victoria, Kings Hebrew stated that Charles decided to escape during an alliance. The mansion house was completed in 1882, the second in its bridge are omitted, while closing is the proton reticulum composed below it aims, such that it is the blurring of appearing on any well-paid type of box printer.

نظر LLM های امروزی:

این متن از نظر معنایی بسیار گیج کننده و به هم ریخته است. به نظر می رسد ترکیبی از موضوعات نامرتب است:

- تاریخ و جغرافیای نامربوط: "دنور شمالی" در آمریکا با "سرویس های اطلاعاتی ایرلندی" در منطقه مدیترانه ارتباط منطقی ندارد.
- اشخاص و مکان های مبهم: "چارلز"، "کینگز عبری"، "ویکتوریا" - بدون زمینه مشخص.
- جملات متناقض: "عمارت در ۱۸۸۲ کامل شد" با "پل دوم حذف شده" ارتباطی ندارد.
- اصطلاحات فنی بی ربط: "پروتون رتیکولوم" (یک اصطلاح علمی) با "چاپگر جعبه" مرتبط نیست.
- به نظر می رسد این متن ممکن است:
 - توسط هوش مصنوعی تولید شده باشد
 - حاصل ترجمه ماشینی ضعیف باشد
 - یک تمرین نوشتاری سورئال باشد
 - متنی باشد که عمدا بی معنی ساخته شده

Symmetrically connected networks

- These are like recurrent networks, but the connections between units are symmetrical (they have the same weight in both directions).
 - John Hopfield (and others) realized that symmetric networks are much easier to analyze than recurrent networks.
 - They are also more restricted in what they can do. because they obey an energy function.
 - For example, they cannot model cycles.
- Symmetrically connected nets without hidden units are called “Hopfield nets”.

Symmetrically connected networks with hidden units

- These are called “Boltzmann machines”.
 - They are much more powerful models than Hopfield nets.
 - They are less powerful than recurrent neural networks.
 - They have a beautifully simple learning algorithm.
- We will cover Boltzmann machines towards the end of the course.

Modern Solution

1. Long Short-Term Memory (LSTM)

Introduced by Hochreiter & Schmidhuber (1997). Uses **gating mechanisms**:

LSTM Equations:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (\text{Forget gate})$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (\text{Input gate})$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad (\text{Candidate memory})$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (\text{Cell state})$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (\text{Output gate})$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (\text{Hidden state})$$

Why it works: The **cell state** $\mathbf{c}_t$ creates a "constant error carousel" where gradients can flow unchanged when gates are open.

Modern Solution

2. Gated Recurrent Unit (GRU)

Simpler alternative by Cho et al. (2014):

GRU Equations:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (\text{Update gate})$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (\text{Reset gate})$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (\text{Candidate hidden})$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad (\text{Hidden state})$$

Question & Answer
